

编译原理

第十讲：中间代码翻译

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 5 月 14 日



福州大学
FUZHOU UNIVERSITY 1958

目录

① 中间代码

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

④ 控制流与回填

⑤ 小结与作业



The logo of Fuzhou University is a circular seal. The outer ring contains the text 'FUZHOU UNIVERSITY' at the top and '福州' at the bottom. Inside the ring, the large Chinese character '福' (Fú) is prominently displayed. Below '福', the year '1958' is visible, along with the characters '大學' (University).

- ① 中间代码
 - 中间代码概述
 - 常见形式
- ② 语法制导定义 (SDD)
- ③ 语法制导翻译 (SDT)
- ④ 控制流与回填
- ⑤ 小结与作业

- ## 中间代码概述
- ### 常见形式

① 中间代码
中间代码概述
常见形式

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

④ 控制流与回填

⑤ 小结与作业

Navigation icons: back, forward, search, etc.

- ## 中间代码概述
- ### 常见形式

Representation, IR) 是源程序在编译过程中的一个中间表示, 位于源程序与机器码之间。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

Representation, IR) 是源程序在编译过程中的中间表示，位于源语言与机器码之间。

源语言) 与后端 (面向目标机器) 之间。

”，再由各方言专家转译。

译成英语再翻译成其他语言。

- **定义：**中间代码（Intermediate Representation, IR）是源程序在编译过程中的一种**中间表示形式**，介于高级语言与机器码之间。
- **地位：**它处于编译器前端（处理源语言）与后端（面向目标机器）之间。
 - 就像先把中文翻译成“世界语”，再由各方言专家转译。
 - 联合国大会上小语种通常先翻译成英语再翻译成其他语言。

Representation, IR) 是源程序在编译过程中的
与机器码之间。
源语言) 与后端 (面向目标机器) 之间。
”, 再由各方言专家转译。
译成英语再翻译成其他语言。

- Representation, IR) 是源程序在编译过程中的
与机器码之间。
源语言) 与后端 (面向目标机器) 之间。
”, 再由各方言专家转译。
译成英语再翻译成其他语言。

Representation, IR) 是源程序在编译过程中的
与机器码之间。
源语言) 与后端 (面向目标机器) 之间。
”, 再由各方言专家转译。
译成英语再翻译成其他语言。

便于适配多种目标架构。

- ① 可移植性：前端与后端分离，便于适配多种目标架构。

Representation, IR) 是源程序在编译过程中的
与机器码之间。
源语言) 与后端 (面向目标机器) 之间。
”, 再由各方言专家转译。
译成英语再翻译成其他语言。
便于适配多种目标架构。
码优化提供结构化支持。

- ① 可移植性：前端与后端分离，便于适配多种目标架构。
- ② 可优化性：为与机器无关的代码优化提供结构化支持。

Representation, IR) 是源程序在编译过程中的
与机器码之间。
源语言) 与后端 (面向目标机器) 之间。
”, 再由各方言专家转译。
译成英语再翻译成其他语言。

便于适配多种目标架构。
码优化提供结构化支持。

耦合度。

- ① **可移植性**：前端与后端分离，便于适配多种目标架构。
- ② **可优化性**：为与机器无关的代码优化提供结构化支持。
- ③ **模块化**：降低编译器各阶段的耦合度。

编
后

The background of the slide features a large, faint, circular seal of Fuzhou University. The seal contains the university's name in English, "FUZHOU UNIVERSITY", around the top edge and in Chinese, "福州大学", around the bottom edge. In the center of the seal is a large, stylized Chinese character "福" (Fú), which means "blessing" or "good fortune". Below the character "福" is the year "1958". The seal is rendered in a light red or pink color, matching the overall theme of the slide.

解决 $M \times N$ 的复杂度

- 没有 IR: 需编写 $M \times N$ 个完整编译器。
- 引入 IR: 只需 M 个前端 + N 个后端, 工作量降为 $M + N$ 。

逻辑架构的解耦

- 将语言相关的语义分析与机的代码生成彻底分离。
- 前端开发者无需懂汇编，后者无需关心源语言类型系统

解决 $M \times N$ 的复杂度

- 没有 IR: 需编写 $M \times N$ 个完整编译器。
- 引入 IR: 只需 M 个前端 + N 个后端, 工作量降为 $M + N$ 。

逻辑架构的解耦

- 将语言相关的语义分析与机器相关的代码生成彻底分离。
- 前端开发者无需懂汇编，后端开发者无需关心源语言类型系统。

逻辑架构的解耦

- 将语言相关的语义分析与机的代码生成彻底分离。
- 前端开发者无需懂汇编，后者无需关心源语言类型系统

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

A large, faint watermark of Fuzhou University's logo is centered on the slide. The logo is circular, featuring the university's name in English 'FUZHOU UNIVERSITY' around the top and in Chinese '福州大学' around the bottom, with the year '1958' at the bottom center. In the center of the logo is a large white Chinese character '福' (Fú).

核心优势

- 
- A large, faint watermark of Fuzhou University's logo is centered on the slide. The logo is circular, featuring the university's name in English 'FUZHOU UNIVERSITY' around the top and in Chinese '福州大学' around the bottom, with the year '1958' at the bottom center. In the center of the logo is a large white Chinese character '福' (Fú).

主要挑战

- 编译速度损耗：多阶段转换与中间代码的存储开销与编译时间。
- 信息丢失：高层语义（如对象生命周期、函数调用）可能被降级为指针偏移量。
- 设计难度：兼顾各种语言特性与硬件约束，实现高效的中间代码生成与优化，任务艰巨且复杂。

7 / 103

引入中间代码 (IR)
较低 ($M + N$)
整体优化效果更佳且通用
较慢 (多层转换)
模块化, 易于维护
LLVM, GCC (GIMPLE), JVM (Bytecode)

多级 IR 策略:

High-Level IR $\rightarrow$ Low-Level IR.

[illegible]

- High-Level IR $\rightarrow$ Mid-Level IR $\rightarrow$ Low-Level IR.

IR 的层次: H / M / L IR

- **高层次 IR (HIR):** 准确表达源语言语义，用于语法错误检查、符号依赖分析、代码重构。

示例: `float a[10][20]; a[i][j+2];`

HIR

```
t1=a[i,j+2];
```

语言语义，用于语法错误检查、符号依赖分析

和目标机器均无关，支持通用优化（常量传播

j+2];

```
*20; t3=t1+t2;  
addr a;  
=*t6;
```

- 示例: `float a[10][20]; a[i][j+2];`

HIR	MIR
t1=a[i,j+2];	t1=j+2; t2=i*20; t3=t1+t2; t4=4*t3; t5=addr a; t6=t5+t4; t7=*t6;

语言语义，用于语法错误检查、符号依赖分析和目标机器均无关，支持通用优化（常量传播、死代码消除等）

寄存器指令一一对应，便于做架构相关优化（寄存器分配）

	LIR
<pre>r3=fp-8; t1=fp-20; t3=t1+t2; addr a; r4=r3*20; r5=r4+r2; r6=4*r5; r7=fp-816; f1=[r7+r6];</pre>	<pre>r1=[fp-4]; r2=[r1+2]; r3=[fp-8]; r4=r3*20; r5=r4+r2; r6=4*r5; r7=fp-816; f1=[r7+r6];</pre>

- 示例: `float a[10][20]; a[i][j+2];`

HIR	MIR	LIR
t1=a[i,j+2];	t1=j+2; t2=i*20; t3=t1+t2; t4=4*t3; t5=addr a; t6=t5+t4; t7=*t6;	r1=[fp-4]; r2=[r1+2]; r3=[fp-8]; r4=r3*20; r5=r4+r2; r6=4*r5; r7=fp-816; f1=[r7+r6];

目录

① 中间代码

中间代码概述

常见形式

逆波兰式

三地址码

树形 IR 与 DAG

LLVM IR

各种中间表示的比较

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

④ 控制流与回填

⑤ 小结与作业

逆波兰式 (Reverse Polish Notation, RPN)

- 由波兰数学家扬·武卡谢维奇于 1920 年提出。



逆波兰式 (Reverse Polish Notation, RPN)

- 由波兰数学家扬·武卡谢维奇于 1920 年提出。
- 核心特征：运算符位于所有相关操作数之后。
 - 例：中缀表达式 $3 + 4$ 的逆波兰式为 $3\ 4\ +$ 。

更多示例

$$a + b * c \Rightarrow a\ b\ c\ * +$$

$$(a + b) * c \Rightarrow a\ b\ +\ c\ *$$

逆波兰式 (Reverse Polish Notation, RPN)

- 由波兰数学家扬·武卡谢维奇于 1920 年提出。
- 核心特征：运算符位于所有相关操作数之后。
 - 例：中缀表达式 $3 + 4$ 的逆波兰式为 $3\ 4\ +$ 。
- 完全消除了括号需求，运算符优先级由位置隐式确定。

更多示例

$$a + b * c \Rightarrow a\ b\ c\ *\ +$$

$$(a + b) * c \Rightarrow a\ b\ +\ c\ *$$

逆波兰式的生成过程（栈算法）

算法步骤

- ① 操作数：直接输出。
- ② 左括号 '('：压栈。
- ③ 右括号 ')'：弹栈输出直到 '('。
- ④ 运算符：弹栈输出所有优先级 $\geq$ 当前运算符的元素，然后压栈当前运算符。
- ⑤ 结束：弹空栈中所有元素。

逆波兰式的生成过程（栈算法）

算法步骤

- ① 操作数：直接输出。
- ② 左括号 '('：压栈。
- ③ 右括号 ')'：弹栈输出直到 '('。
- ④ 运算符：弹栈输出所有优先级 $\geq$ 当前运算符的元素，然后压栈当前运算符。
- ⑤ 结束：弹空栈中所有元素。

模拟执行算法

- ① 从左到右扫描表达式。
- ② 操作数：压栈。
- ③ 运算符：弹出栈顶两个操作数，执行运算后将结果压栈。
- ④ 栈顶即为最终结果。

后缀式生成示例: $2 + 3 * 4$

输入: 2 + 3 * 4

输出:

符号栈:



符号栈:

后缀式生成示例: $2 + 3 * 4$

输入:

3 * 4

输出: 2

符号栈: +

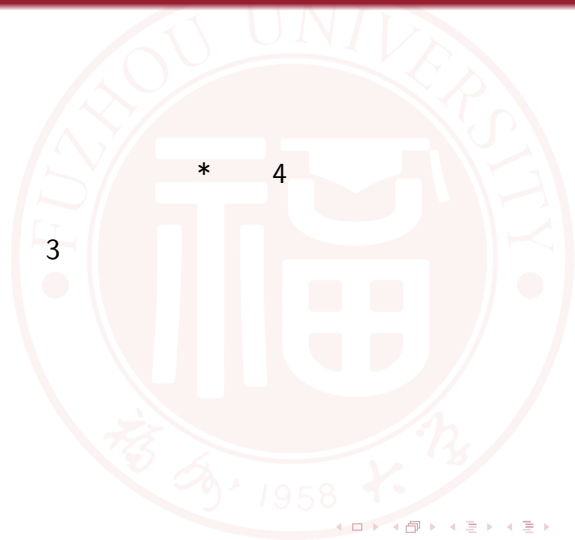


后缀式生成示例: $2 + 3 * 4$

输入:

输出: 2

符号栈: +



后缀式生成示例: $2 + 3 * 4$

输入:

4

输出: 2

3

符号栈: +

*

3 4 *

福州大学 1958

Navigation icons: back, forward, search, etc.

符号栈: + *

3 4 *

福

福州 1958

输出: 2 3 4 *

符号栈: +

3 4 * +

输出: 2 3 4 * +

符号栈:

逆波兰式的优缺点

优点

- 无需括号，语法无歧义。
- 求值算法简单，利于栈机实现（如 JVM 字节码）。
- 适合表达式的线性表示与快速解析。

缺点

- 人工可读性差，难以表达控制流。
- 无法显式表达公共子表达式，不利于优化。
- 对并行性表达较弱。

三地址码 (Three-Address Code, TAC)

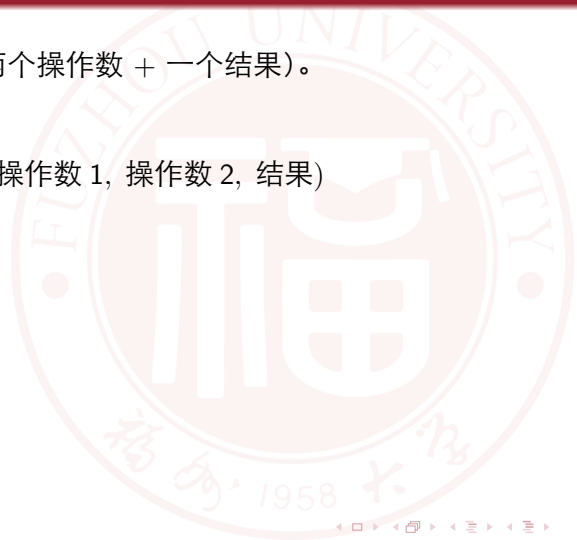
- 每条指令最多包含三个地址 (两个操作数 + 一个结果)。



三地址码 (Three-Address Code, TAC)

- 每条指令最多包含三个地址 (两个操作数 + 一个结果)。
- 基本形式可视为四元组:

(运算符, 操作数 1, 操作数 2, 结果)



三地址码 (Three-Address Code, TAC)

- 每条指令最多包含三个地址 (两个操作数 + 一个结果)。
- 基本形式可视为四元组:

(运算符, 操作数 1, 操作数 2, 结果)

- 通过引入临时变量, 将复杂表达式分解为简单的原子操作序列。

三地址码 (Three-Address Code, TAC)

- 每条指令最多包含三个地址 (两个操作数 + 一个结果)。
- 基本形式可视为四元组:

(运算符, 操作数 1, 操作数 2, 结果)

- 通过引入临时变量, 将复杂表达式分解为简单的原子操作序列。

示例: $a = b + c * d$

$t1 = c * d$

$a = b + t1$

三地址码的常见形式

- 赋值指令：



三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)



三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)



三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)



三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：



三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：
 - $x = y[i], \quad x[i] = y^1$

¹实际实现需参考定义

三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：
 - $x = y[i], \quad x[i] = y^1$
 - $x = \&y, \quad x = *y, \quad *x = y$

¹实际实现需参考定义

三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：
 - $x = y[i], \quad x[i] = y^1$
 - $x = \&y, \quad x = *y, \quad *x = y$
- 转移指令：

¹实际实现需参考定义

三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：
 - $x = y[i], \quad x[i] = y^1$
 - $x = \&y, \quad x = *y, \quad *x = y$
- 转移指令：
 - `goto L` (无条件跳转)

¹实际实现需参考定义

三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：
 - $x = y[i], \quad x[i] = y^1$
 - $x = \&y, \quad x = *y, \quad *x = y$
- 转移指令：
 - `goto L` (无条件跳转)
 - `if x relop y goto L` (条件跳转)

¹实际实现需参考定义

三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：
 - $x = y[i], \quad x[i] = y^1$
 - $x = \&y, \quad x = *y, \quad *x = y$
- 转移指令：
 - $\text{goto } L$ (无条件跳转)
 - $\text{if } x \text{ relop } y \text{ goto } L$ (条件跳转)
- 过程调用：

¹实际实现需参考定义

三地址码的常见形式

- 赋值指令：
 - $x = y \text{ op } z$ (双目运算)
 - $x = \text{op } y$ (单目运算)
 - $x = y$ (复制)
- 数组与指针：
 - $x = y[i], \quad x[i] = y^1$
 - $x = \&y, \quad x = *y, \quad *x = y$
- 转移指令：
 - `goto L` (无条件跳转)
 - `if x relop y goto L` (条件跳转)
- 过程调用：
 - `param x, call p, n, y = call p, n`

¹实际实现需参考定义

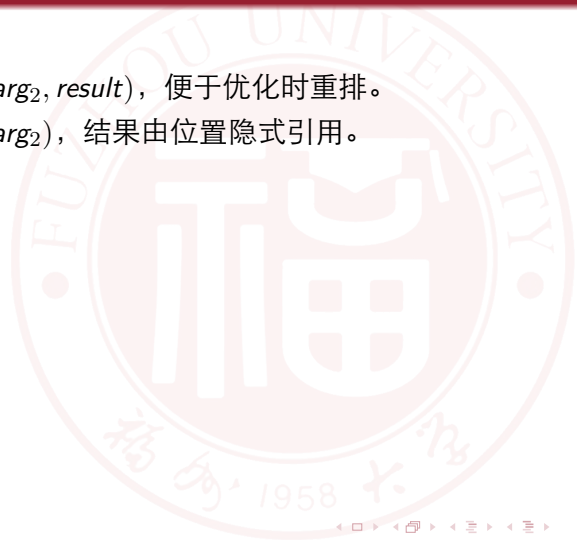
三地址码的存储表示

- 四元式：四字段记录 ($op, arg_1, arg_2, result$)，便于优化时重排。



三地址码的存储表示

- 四元式：四字段记录 ($op, arg_1, arg_2, result$)，便于优化时重排。
- 三元式：三字段记录 (op, arg_1, arg_2)，结果由位置隐式引用。



三地址码的存储表示

- 四元式：四字段记录 ($op, arg_1, arg_2, result$)，便于优化时重排。
- 三元式：三字段记录 (op, arg_1, arg_2)，结果由位置隐式引用。
- 间接三元式：三元式表 + 间接码表（指针数组），兼顾空间效率与优化灵活性。

三地址码的存储表示

- 四元式：四字段记录 ($op, arg_1, arg_2, result$)，便于优化时重排。
- 三元式：三字段记录 (op, arg_1, arg_2)，结果由位置隐式引用。
- 间接三元式：三元式表 + 间接码表（指针数组），兼顾空间效率与优化灵活性。

$a = b * (-c)$ 的四元式

序号	op	arg_1	arg_2	result
(0)	uminus	c	—	t1
(1)	*	b	t1	t2
(2)	=	t2	—	a

三地址码到汇编的对应

三地址码	x86 汇编	ARM 汇编
$x = y + z$	<code>mov eax, [y]; add eax, [z]; mov [x], eax</code>	<code>LDR r0, [y]; LDR r1, [z]; ADD r0, r0, r1; STR r0, [x]</code>
<code>goto L</code>	<code>jmp L</code>	<code>B L</code>
<code>if a < b goto L</code>	<code>cmp eax, [b]; jl L</code>	<code>CMP r0, r1; BLT L</code>
$x = y[i]$	<code>mov esi, [i]; mov eax, [y+esi*4]</code>	<code>LDR r0, =y; LDR r1, [i]; LDR r0, [r0, r1, LSL #2]</code>

综合示例

构造下述语句的三地址码、四元式、三元式、间接三元式

$X = (A + B) * C;$ $Y = D \uparrow (A + B);$

综合示例

构造下述语句的三地址码、四元式、三元式、间接三元式

$$X = (A + B) * C; \quad Y = D \uparrow (A + B);$$

三地址码:

$t1 = A + B;$

$X = t1 * C;$

$Y = D \uparrow t1;$

综合示例

构造下述语句的三地址码、四元式、三元式、间接三元式

$X = (A + B) * C;$ $Y = D \uparrow (A + B);$

三地址码:

$t1 = A + B;$

$X = t1 * C;$

$Y = D \uparrow t1;$

四元式:

序号	op	arg1	arg2	result
(0)	+	A	B	t1
(1)	*	C	t1	t2
(2)	=	t2	—	X
(3)	↑	D	t1	t3
(4)	=	t3	—	Y

综合示例

构造下述语句的三地址码、四元式、三元式、间接三元式

$X = (A + B) * C;$ $Y = D \uparrow (A + B);$

三地址码:

$t1 = A + B;$

$X = t1 * C;$

$Y = D \uparrow t1;$

四元式:

序号	op	arg1	arg2	result
(0)	+	A	B	t1
(1)	*	C	t1	t2
(2)	=	t2	—	X
(3)	↑	D	t1	t3
(4)	=	t3	—	Y

三元式:

序号	op	arg1	arg2
(0)	+	A	B
(1)	*	C	(0)
(2)	=	X	(1)
(3)	↑	D	(0)
(4)	=	Y	(3)

综合示例

构造下述语句的三地址码、四元式、三元式、间接三元式

 $X = (A + B) * C; \quad Y = D \uparrow (A + B);$

三地址码:

 $t1 = A + B;$ $X = t1 * C;$ $Y = D \uparrow t1;$

四元式:

序号	op	arg1	arg2	result
(0)	+	A	B	t1
(1)	*	C	t1	t2
(2)	=	t2	—	X
(3)	↑	D	t1	t3
(4)	=	t3	—	Y

三元式:

序号	op	arg1	arg2
(0)	+	A	B
(1)	*	C	(0)
(2)	=	X	(1)
(3)	↑	D	(0)
(4)	=	Y	(3)

间接码表:

序号	Statement
(10)	(0)
(11)	(1)
(12)	(3)
(13)	(2)
(14)	(4)

三地址码的优缺点

优点

- 结构简单、形式统一，易于生成与处理。
- 机器无关性强，便于跨平台移植。
- 四元式形式便于优化（公共子表达式删除、循环优化）。
- 是现代编译器中最广泛使用的 IR 形式之一。

三地址码的优缺点

优点

- 结构简单、形式统一，易于生成与处理。
- 机器无关性强，便于跨平台移植。
- 四元式形式便于优化（公共子表达式删除、循环优化）。
- 是现代编译器中最广泛使用的 IR 形式之一。

缺点

- 引入大量临时变量，占用额外存储空间。
- 线性结构丢失了表达式原有的树形层次信息。
- 控制流依赖显式标号，高层结构不明显。

练习 1: 算术表达式与临时变量

题目

将以下赋值语句转换为三地址码，注意处理负号优先级和括号。

$$x = -a + (b * c) / (d - e * f)$$

练习 1: 算术表达式与临时变量

题目

将以下赋值语句转换为三地址码，注意处理负号优先级和括号。

$$x = -a + (b * c) / (d - e * f)$$

参考答案

```
t1 = uminus a
t2 = b * c
t3 = e * f
t4 = d - t3
t5 = t2 / t4
t6 = t1 + t5
x = t6
```


练习 1: 算术表达式与临时变量

题目

将以下赋值语句转换为三地址码，注意处理负号优先级和括号。

$$x = -a + (b * c) / (d - e * f)$$

参考答案

```
t1 = uminus a
t2 = b * c
t3 = e * f
t4 = d - t3
t5 = t2 / t4
t6 = t1 + t5
x = t6
```

要点

- 单目运算符: $-a$ 需独立占用一条指令 (uminus)。
- 确认优先级: 先乘除、后加减, 括号最优先。

练习 2: 短路逻辑与跳转指令

题目

将以下控制流语句转换为三地址码。

```
if (a < b || c == d && e > f)
    x = 1;
else x = 0;
```



练习 2: 短路逻辑与跳转指令

题目

将以下控制流语句转换为三地址码。

```
if (a < b || c == d && e > f)
    x = 1;
else x = 0;
```

重点

- 短路求值：|| 左侧为真则跳过右侧。
- 优先级：&& 高于 ||。



练习 2: 短路逻辑与跳转指令

题目

将以下控制流语句转换为三地址码。

```
if (a < b || c == d && e > f)
    x = 1;
else x = 0;
```

重点

- 短路求值: || 左侧为真则跳过右侧。
- 优先级: && 高于 ||。

参考答案

```
    if a < b goto L2
    goto L3
L3:   if c == d goto L4
    goto L1
L4:   if e > f goto L2
    goto L1
L2:   x = 1
    goto L5
L1:   x = 0
L5:
```

练习 3: 多维数组寻址与指针

题目

数组 A 声明为 `int A[10][20]`, `int` 占 4 字节。生成以下语句的 TAC:

$$y = A[i][j + 2] * *p$$

练习 3: 多维数组寻址与指针

题目

数组 A 声明为 `int A[10][20]`, `int` 占 4 字节。生成以下语句的 TAC:

$$y = A[i][j + 2] * *p$$

要点解析

- 二维数组寻址: $\text{base} + (i \times n + j) \times w$, 其中 $n = 20, w = 4$ 。
- 间接寻址: `*p` 取指针 `p` 所指向地址的内容。

练习 3: 多维数组寻址与指针

题目

数组 A 声明为 `int A[10][20]`, `int` 占 4 字节。生成以下语句的 TAC:

$$y = A[i][j + 2] * *p$$

要点解析

- 二维数组寻址: $\text{base} + (i \times n + j) \times w$, 其中 $n = 20, w = 4$ 。
- 间接寻址: `*p` 取指针 `p` 所指向地址的内容。

参考答案

```
// A[i][j + 2]
t1 = i * 20
t2 = j + 2
t3 = t1 + t2
t4 = t3 * 4
t5 = A[t4]
// *p
t6 = *p
// 最终乘法
t7 = t5 * t6
y = t7
```

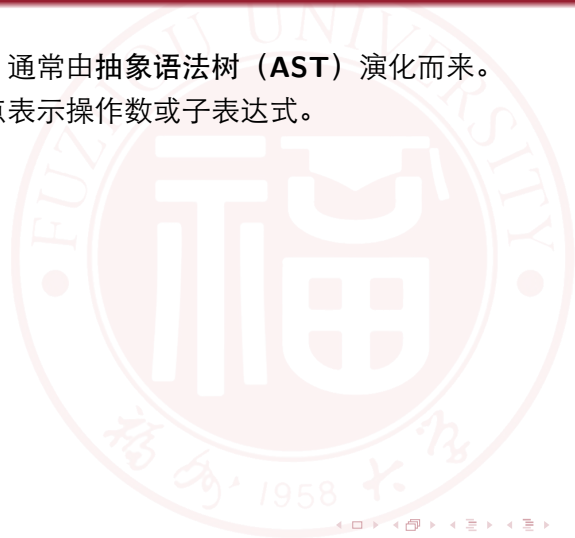
树形 IR (Tree-based IR)

- 以树形数据结构表示程序结构，通常由抽象语法树 (AST) 演化而来。



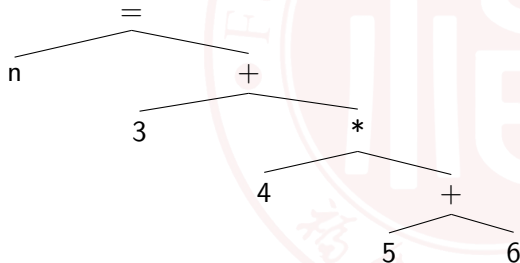
树形 IR (Tree-based IR)

- 以树形数据结构表示程序结构，通常由抽象语法树 (**AST**) 演化而来。
- 每个节点代表一个操作，子节点表示操作数或子表达式。



树形 IR (Tree-based IR)

- 以树形数据结构表示程序结构，通常由抽象语法树 (AST) 演化而来。
- 每个节点代表一个操作，子节点表示操作数或子表达式。
- 例： $n = 3 + 4 * (5 + 6)$ 的 AST:



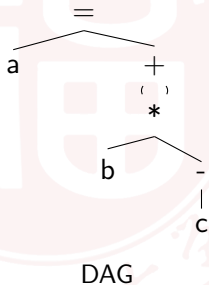
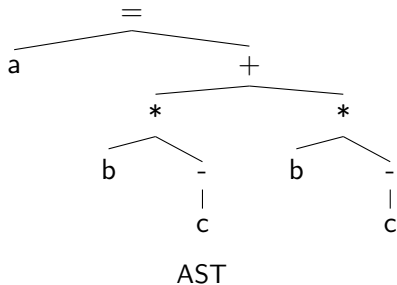
有向无环图 (DAG)

- DAG 是对 AST 的改进：共享公共子表达式节点。



有向无环图 (DAG)

- DAG 是对 AST 的改进：共享公共子表达式节点。
- 例： $a = b * (-c) + b * (-c)$



树形 IR 的优缺点

优点

- 层次结构清晰，直观反映程序语法结构。
- 便于语法分析、语义检查和局部优化（常量折叠）。
- DAG 可自动消除冗余计算（公共子表达式）。

树形 IR 的优缺点

优点

- 层次结构清晰，直观反映程序语法结构。
- 便于语法分析、语义检查和局部优化（常量折叠）。
- DAG 可自动消除冗余计算（公共子表达式）。

缺点

- 复杂控制流（分支、循环）难以用单棵树表达。
- 跨作用域分析与全局优化能力较弱。
- DAG 的构建与维护成本较高。

LLVM IR

- LLVM IR 是 LLVM 编译器基础设施使用的低层次中间表示。



LLVM IR

- LLVM IR 是 LLVM 编译器基础设施使用的低层次中间表示。
- 采用静态单赋值 (**SSA**) 形式：每个变量只被赋值一次。

LLVM IR

- LLVM IR 是 LLVM 编译器基础设施使用的低层次中间表示。
- 采用静态单赋值 (**SSA**) 形式：每个变量只被赋值一次。
- 基本组织单位是模块 (**Module**)，包含函数、全局变量等。

LLVM IR

- LLVM IR 是 LLVM 编译器基础设施使用的低层次中间表示。
- 采用静态单赋值 (**SSA**) 形式：每个变量只被赋值一次。
- 基本组织单位是模块 (**Module**)，包含函数、全局变量等。
- 指令采用类汇编的三地址格式，但带有**高级类型信息**。

LLVM IR 示例: $a = b + c * d$

```
%t1 = mul i32 %c, %d  
%a  = add i32 %b, %t1
```

LLVM IR 的关键特性

- **SSA 形式:**



LLVM IR 的关键特性

- **SSA 形式：**
 - 简化数据流分析（到达定义、常量传播）。



LLVM IR 的关键特性

- **SSA 形式:**
 - 简化数据流分析 (到达定义、常量传播)。
 - 分支合并处使用 $\text{phi}(\Phi)$ 指令选择不同路径的值。

LLVM IR 的关键特性

- **SSA 形式：**
 - 简化数据流分析（到达定义、常量传播）。
 - 分支合并处使用 $\text{phi}(\Phi)$ 指令选择不同路径的值。
- **强类型系统：**显式指定 `i32`、`float`、`ptr` 等类型。

LLVM IR 的关键特性

- **SSA 形式：**
 - 简化数据流分析（到达定义、常量传播）。
 - 分支合并处使用 `phi( $\Phi$ )` 指令选择不同路径的值。
- **强类型系统：** 显式指定 `i32`、`float`、`ptr` 等类型。
- **无限虚拟寄存器：** 以 `%` 开头的局部命名值。

LLVM IR 的关键特性

- **SSA 形式：**
 - 简化数据流分析（到达定义、常量传播）。
 - 分支合并处使用 $\text{phi}(\Phi)$ 指令选择不同路径的值。
- **强类型系统：**显式指定 `i32`、`float`、`ptr` 等类型。
- **无限虚拟寄存器：**以 `%` 开头的局部命名值。
- **结构化控制流：**`br` 指令支持条件与无条件跳转，`ret` 表示返回。

SSA 与 Phi 函数

非 SSA 形式

$x = 1;$

$x = x + 2;$



SSA 与 Phi 函数

非 SSA 形式

```
x = 1;  
x = x + 2;
```

SSA 形式

```
x1 = 1;  
x2 = x1 + 2;
```

SSA 与 Phi 函数

非 SSA 形式

```
x = 1;  
x = x + 2;
```

SSA 形式

```
x1 = 1;  
x2 = x1 + 2;
```

Phi 函数示例

```
if (cond) { x1 = 1; } else { x2 = 2; }  
x3 = Phi(x1, x2); // 根据路径选择 x1 或 x2  
y1 = x3 + 3;
```

SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法:



SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块的支配前沿 (合并控制流的边界)。

SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Phi 函数。

dominance Frontier) 算法：
合并控制流的边界)。
入 Phi 函数。

- dominance Frontier) 算法：
合并控制流的边界)。
入 Phi 函数。

SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Phi 函数。
- 变量重命名：
 - 遍历支配树, 为每个变量赋予唯一版本号 ($x \rightarrow x_1, x_2, x_3$)。

SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Phi 函数。
- 变量重命名：
 - 遍历支配树, 为每个变量赋予唯一版本号 ($x \rightarrow x_1, x_2, x_3$)。
 - 替换变量的引用为对应版本。

SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Phi 函数。
- 变量重命名：
 - 遍历支配树，为每个变量赋予唯一版本号 ($x \rightarrow x_1, x_2, x_3$)。
 - 替换变量的引用为对应版本。
- 处理非 **SSA** 场景：

SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Phi 函数。
- 变量重命名：
 - 遍历支配树, 为每个变量赋予唯一版本号 ($x \rightarrow x_1, x_2, x_3$)。
 - 替换变量的引用为对应版本。
- 处理非 **SSA** 场景：
 - 内存操作 (如 $*p = 10$) 需转换为显式加载/存储。

SSA 的实现要点

- 插入 **Phi** 函数——支配前沿 (Dominance Frontier) 算法：
 - 计算每个基本块的支配前沿 (合并控制流的边界)。
 - 在支配前沿的入口处为变量插入 Phi 函数。
- 变量重命名：
 - 遍历支配树，为每个变量赋予唯一版本号 ($x \rightarrow x_1, x_2, x_3$)。
 - 替换变量的引用为对应版本。
- 处理非 **SSA** 场景：
 - 内存操作 (如 $*p = 10$) 需转换为显式加载/存储。
 - 函数调用保守假设全局变量可能被修改。

LLVM IR 的优缺点

优点

- SSA 形式极大简化了数据流分析与优化。
- 类型安全、文档丰富、生态成熟。
- 与多种前端和后端无缝衔接。
- 支持丰富的优化 Pass（死代码消除、循环优化、向量化等）。

LLVM IR 的优缺点

优点

- SSA 形式极大简化了数据流分析与优化。
- 类型安全、文档丰富、生态成熟。
- 与多种前端和后端无缝衔接。
- 支持丰富的优化 Pass（死代码消除、循环优化、向量化等）。

缺点

- 学习曲线陡峭，指令集较庞大。
- 对人类不友好，调试需借助工具。
- 作为低层 IR，部分高层语义表达较繁琐。

四种中间表示的综合比较

特性	逆波兰式	三地址码	树形 IR	LLVM IR
结构形式	线性序列	线性序列	树 / DAG	线性 SSA
可读性	较差	中等	好	较差
控制流表达	弱	强 (显式标号)	弱	强 (br/phi)
优化支持	弱	中等 (四元式佳)	局部优化好	全局优化极佳
应用场景	栈机、JVM	教学、传统编译器	前端分析	工业级编译器
典型系统	Forth, HP 计算机	GCC GIMPLE (早期)	虎书 Tiger	LLVM, Clang

- ① 中间代码
- ② 语法制导定义 (SDD)
 - 概念
 - 属性分类
 - 属性求值顺序
- ③ 语法制导翻译 (SDT)
- ④ 控制流与回填
- ⑤ 小结与作业

① 中间代码

② 语法制导定义 (SDD)

- 概念
- 属性分类
- 属性求值顺序

③ 语法制导翻译 (SDT)

④ 控制流与回填

⑤ 小结与作业

Navigation icons: back, forward, search, etc.

nd Definition, SDD) 是对上下文无关文法的

- **语法制导定义 (Syntax-Directed Definition, SDD)** 是对上下文无关文法的推广。

and Definition, SDD) 是对上下文无关文法的
(attribute)。

- and Definition, SDD) 是对上下文无关文法的
(tribute)。

and Definition, SDD) 是对上下文无关文法的
tribute)。
semantic rules), 用于计算属性值。

- **语法制导定义 (Syntax-Directed Definition, SDD)** 是对上下文无关文法的推广。
- 每个文法符号关联一组属性 (attribute)。
- 每条产生式关联一组语义规则 (semantic rules)，用于计算属性值。

ed Definition, SDD) 是对上下文无关文法的
(attribute)。
(semantic rules), 用于计算属性值。
“计算着”在文法的产生式上, 随语法分析同步进行

- 语法制导定义 (Syntax-Directed Definition, SDD) 是对上下文无关文法的推广。
- 每个文法符号关联一组属性 (attribute)。
- 每条产生式关联一组语义规则 (semantic rules), 用于计算属性值。
- 核心思想: 将语义计算规则“附着”在文法的产生式上, 随语法分析同步进行。

定义规则

$$code = E_1.code || E_2.code || op$$

表示任意二元操作符，“||”表示后缀形式的连接

产生式	语义规则
$E \rightarrow E_1 \text{ op } E_2$	$E.code = E_1.code E_2.code \text{op}$

其中, $E.code$ 表示 E 后缀形式, op 表示任意二元操作符, “||”表示后缀形式的连接。

③ 语法制导翻译 (SDT)

- 文法符号: L 、 E 、 T 、 F 、 $digit$
- 属性: val
- 语义规则: 通常写成表达式的形式用于计算文法符号的属性值

- 文法符号: L 、 E 、 T 、 F 、 $digit$
- 属性: val
- 语义规则: 通常写成表达式的形式, 用于计算文法符号的属性值

- 文法符号: L 、 E 、 T 、 F 、 $digit$
- 属性: val
- 语义规则: 通常写成表达式的形式, 用于计算文法符号的属性值

文法符号的属性分类:

- 综合属性 (Synthesized Attribute)
- 继承属性 (Inherited Attribute)

- 文法符号: L 、 E 、 T 、 F 、 $digit$
- 属性: val
- 语义规则: 通常写成表达式的形式, 用于计算文法符号的属性值

文法符号的属性分类:

- 综合属性 (Synthesized Attribute)
- 继承属性 (Inherited Attribute)

A large, faint watermark of the Fuzhou University logo is centered on the slide. The logo is circular, featuring the university's name in English, "FUZHOU UNIVERSITY", around the top and the Chinese characters "福州大学" around the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú). The watermark is light gray and serves as a background element.

综合属性 (Synthesized)

- 根据子节点的属性计算父节点的属性。
- 自底向上传递信息。
- 例：表达式类型、计算结果值。

例

$$E \rightarrow E_1 + T$$

$$E.val = E_1.val + T.val$$

继承属性 (Inherited)

- 根据父节点或兄弟节点的属当前节点的属性。
- 自顶向下或横向传递信息。
- 例：变量类型、相对地址。

例

$$D \rightarrow T \text{ id} ; D_1$$

$$D_1.type = T.type$$

1. *Journal of Management Studies*, 1996, 33, 1, 1-14.

40 / 103

- 在分析树结点 N 上的非终结符 A 的**继承属性**只能通过 **N 的父结点**、**N 的兄弟结点**或 **N 本身**的属性值来定义。

A diagram showing a node D at the top. Two solid black lines branch downwards from D to $T.type$ on the left and $L.inh$ on the right. A red dashed arrow points from $T.type$ to $L.inh$.

- 41 / 103

A large, faint watermark of the Fuzhou University logo is centered on the slide. The logo is circular, featuring the university's name in English, "FUZHOU UNIVERSITY", around the top and the Chinese characters "福州大学" around the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú). Below the characters, the year "1958" is visible. The watermark is light gray and serves as a background element.

语法规则

A large, faint, circular watermark of the Fuzhou University logo is centered on the page. The logo features the university's name in English, "FUZHOU UNIVERSITY", around the top arc and in Chinese, "福州大学", around the bottom arc, with the year "1958" at the bottom. In the center of the logo is a large, stylized Chinese character "福" (Fú).

语法规则

产生式	语义规则
$L \rightarrow E \ n$	$print(E.val)^a$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval^b$

^a虚拟综合属性、副作用

^b终结符只有综合属性，由词法分析器提供

和出现在产生式左边的综合属性都必须提供一
能使用本产生式中文法符号的属性。

- 对出现在产生式**右边**的**继承属性**和出现在产生式**左边**的**综合属性**都必须提供一个**计算规则**。属性计算规则只能使用本产生式中的文法符号的属性。

和出现在产生式**左边**的**综合属性**都必须提供一
使用本产生式中文法符号的属性。

号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到

- 和出现在产生式**左边**的**综合属性**都必须提供一
使用本产生式中文法符号的属性。
- 号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到

例

对于产生式 $F \rightarrow digit$, 左侧文法符号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到。

- 对出现在产生式**右边**的**继承属性**和出现在产生式**左边**的**综合属性**都必须提供一个**计算规则**。属性计算规则只能使用本产生式中的文法符号的属性。

例

对于产生式 $F \rightarrow digit$ ，左侧文法符号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到。

- 出现在产生式**右边**的**综合属性**和出现在产生式**左边**的**继承属性**不由该产生式所给的属性计算规则进行计算，由**其他产生式**的属性计算规则或者由属性计算器的参数提供。

对于产生式 $F \rightarrow digit$, 左侧文法符号 F 的综合属性 val 由 $F.val = digit.lexval$ 得到。

- 43 / 103

考虑非终结符 A 、 B 和 C ， A 有继承属性 a 和综合属性 b ， B 有综合属性 c ， C 有继承属性 d 。

- ① $C.d = B.c + 1$
- ② $A.b = A.a + B.c$
- ③ $B.c = A.a$

计算规则示例

例

考虑非终结符 A 、 B 和 C ， A 有继承属性 a 和综合属性 b ， B 有综合属性 c ， C 有继承属性 d 。

产生式 $A \rightarrow BC$ 不可能有以下规则的哪一个？

- ① $C.d = B.c + 1$
- ② $A.b = A.a + B.c$
- ③ $B.c = A.a$

考虑非终结符 A 、 B 和 C ， A 有继承属性 a 和综合属性 b ， B 有综合属性 c ， C 有继承属性 d 。

- ① $C.d = B.c + 1$
- ② $A.b = A.a + B.c$
- ③ $B.c = A.a$

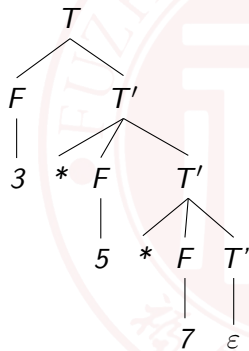
属性 $A.a$ 和 $B.c$ 应用其它产生式的语义规则计算。

定义规则
$T'.inh = F.val$
$T'.val = T'.syn$
$T'.inh = T.inh \times F.val$
$T'.syn = T'_1.syn$
$T'.syn = T'.inh$
$T'.val = digit.lexval$

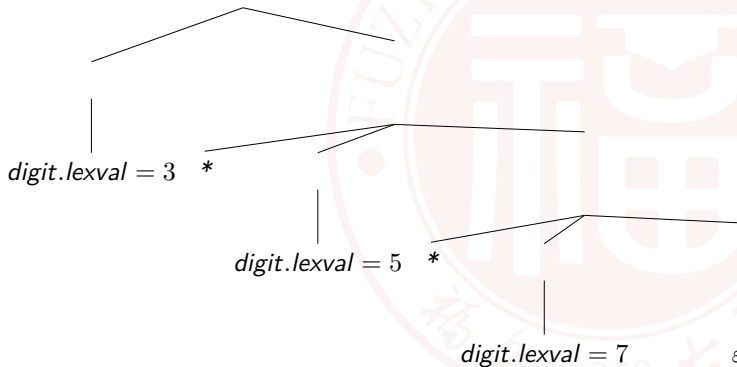
产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

解

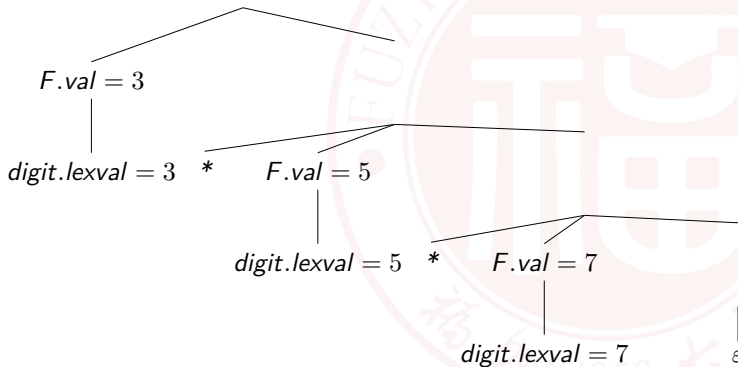
① 画出分析树



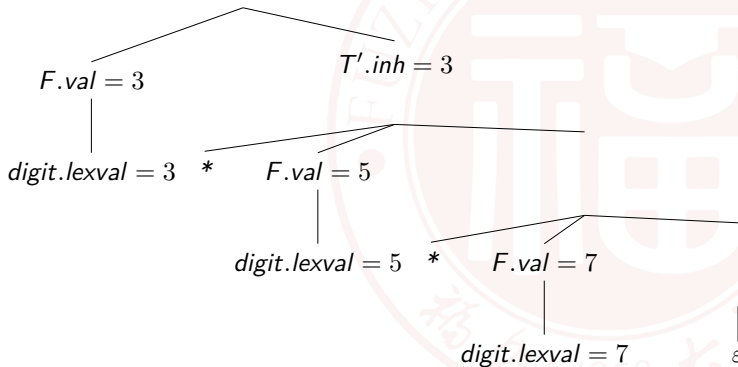
② 计算节点属性



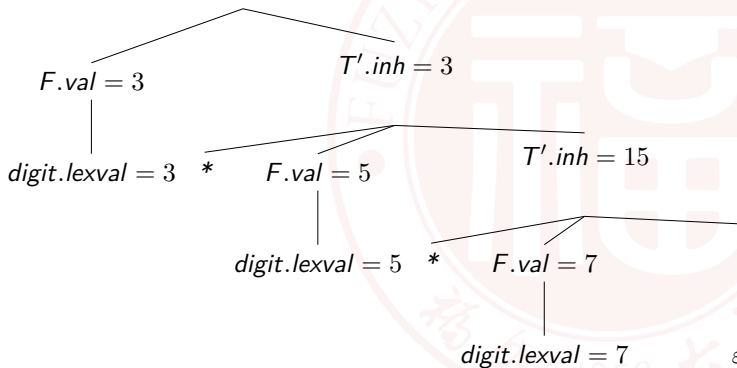
② 计算节点属性



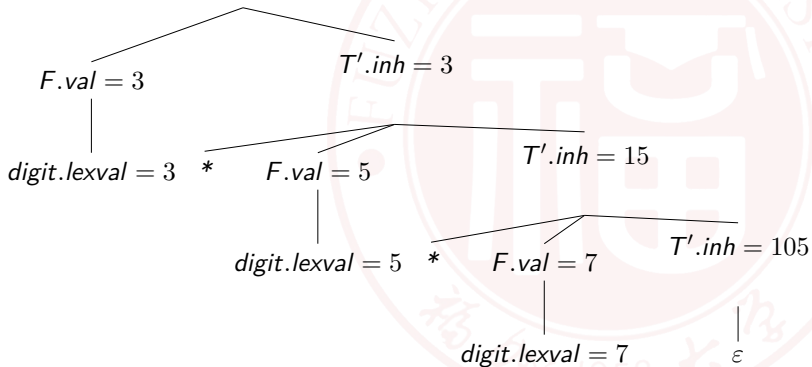
② 计算节点属性



② 计算节点属性

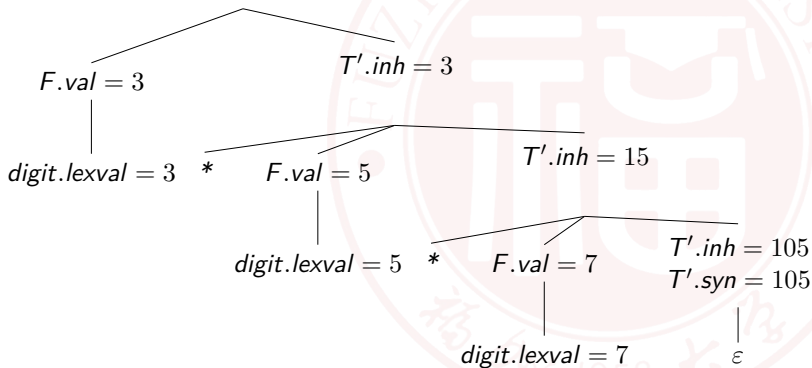


② 计算节点属性



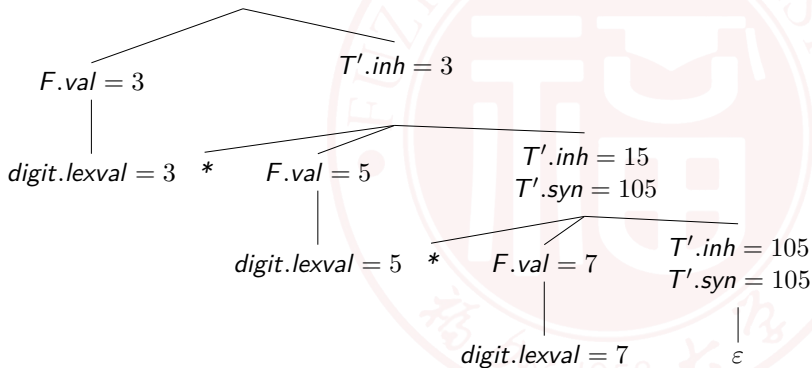
解

② 计算节点属性



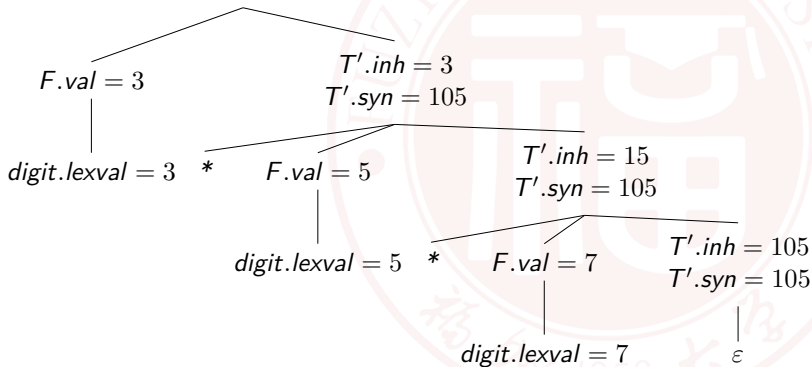
解

② 计算节点属性



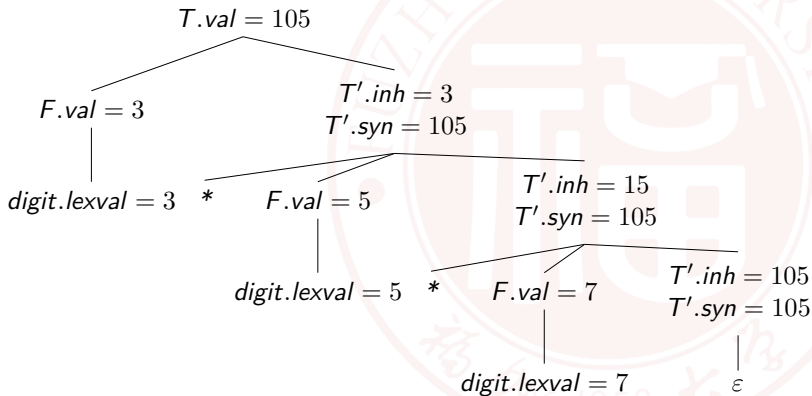
解

② 计算节点属性



解

② 计算节点属性



目录

① 中间代码

② 语法制导定义 (SDD)

概念

属性分类

属性求值顺序

③ 语法制导翻译 (SDT)

④ 控制流与回填

⑤ 小结与作业

属性。对于给定的输入串 x ，若要应用语义规

- 应按照什么顺序计算属性值？

SDD 为 CFG 中的文法符号设置语义属性。对于给定的输入串 x ，若要应用语义规则计算分析树中各结点对应的属性值则

- 应按照什么顺序计算属性值？
 - 语义规则建立了属性之间的依赖关系，在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。

- 应按照什么顺序计算属性值？
 - 语义规则建立了属性之间的依赖关系，在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。

SDD 为 CFG 中的文法符号设置语义属性。对于给定的输入串 x ，若要应用语义规则计算分析树中各结点对应的属性值则

- 应按照什么顺序计算属性值？
 - 语义规则建立了属性之间的依赖关系，在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。
 - 文法开始符号的继承属性应事先给定。

- 应按照什么顺序计算属性值？
 - 语义规则建立了属性之间的依赖关系，在对语法分析树节点的一个属性求值之前，必须首先求出这个属性值所依赖的所有属性值。
 - 文法开始符号的继承属性应事先给定。

依赖图 (Dependency Graph)

- 语法树中各结点的综合属性和继承属性之间的相互依赖关系可以用**依赖图**（有向图）来描述。
- 依赖图指出语法树中各结点属性值的计算顺序。
- 特点：先有文法，后确定语义计算顺序。

依赖图构建算法

```

1  foreach 所有语法分析树的节点  $n$  do
2      | foreach 结点  $n$  对应文法符号的每个属性  $a$  do
3          | 在依赖图中为  $a$  构建一个结点;
4      end
5  end
6  foreach 所有语法分析树的节点  $n$  do
7      | foreach 结点  $n$  的产生式的每条语义规则  $b = f(c_1, c_2, \dots, c_k)$  do
8          | foreach  $c_j$  do
9              | 从结点  $c_j$  到  $n$  构造一条有向边;
10         end
11     end
12 end

```

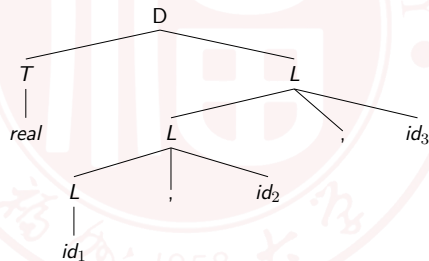
依赖图的示例

例 (SDD 依赖图示例)

考虑产生式和语义规则如下:

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。



依赖图的示例

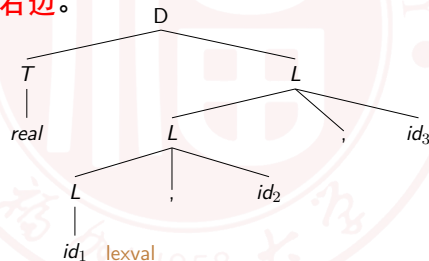
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将继承属性 (**inh**) 放在分析树结点的**左边**，综合属性 (**type**, **lexeme**) 放在分析树结点的**右边**。



例 (SDD 依赖图示例)

为了便于区分，一般将继承属性 (inh) 放在分析树结点的左边，综合属性 (type, lexeme) 放在分析树结点的右边。

```

graph TD
    D --> T
    D --> L1[L]
    T --> real
    L1 --> L2[L]
    L1 --> comma1[,]
    L1 --> id3[id3]
    L2 --> L3[L]
    L2 --> comma2[,]
    L2 --> id2[id2]
    L3 --> id1[id1]
    id1 --- lexval1[lexval]
    id2 --- lexval2[lexval]
    id3 --- lexval3[lexval]
  
```

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

依赖图的示例

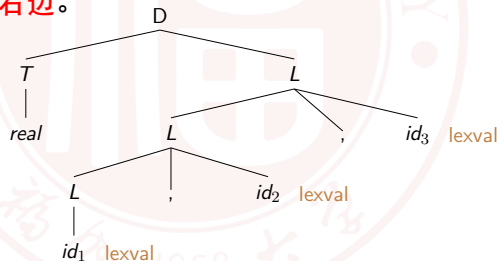
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将继承属性 (**inh**) 放在分析树结点的**左边**，综合属性 (**type**, **lexeme**) 放在分析树结点的**右边**。

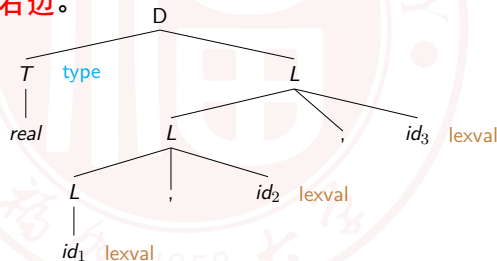


例 (SDD 依赖图示例)

为了便于区分，一般将继承属性 (inh) 放在分析树结点的左边，综合属性 (type, lexeme) 放在分析树结点的右边。

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 *real id₁, id₂, id₃*。



例 (SDD 依赖图示例)

为了便于区分，一般将继承属性 (inh) 放在分析树结点的左边，综合属性 (type, lexeme) 放在分析树结点的右边。

```

graph TD
    D --> T
    D --> L1[L]
    T --> real
    L1 -- inh --> L2[L]
    L1 --> comma1[']
    L1 --> id3[id3]
    L2 --> L3[L]
    L2 --> comma2[']
    L2 --> id2[id2]
    L3 --> id1[id1]
    
```

A set of small navigation icons typically found in Beamer presentations, including symbols for back, forward, search, and other slide controls.

53 / 103

依赖图的示例

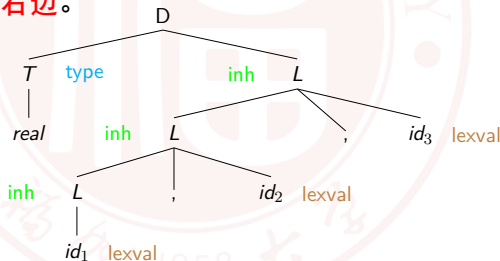
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将继承属性 (**inh**) 放在分析树结点的**左边**，综合属性 (**type**, **lexeme**) 放在分析树结点的**右边**。



53 / 103

依赖图的示例

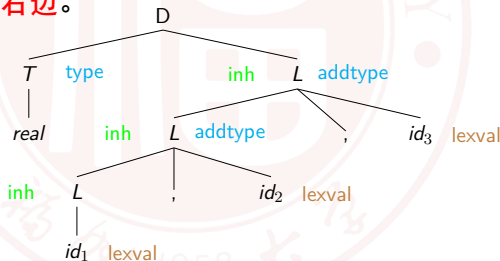
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将继承属性 (**inh**) 放在分析树结点的**左边**，综合属性 (**type**, **lexeme**) 放在分析树结点的**右边**。



依赖图的示例

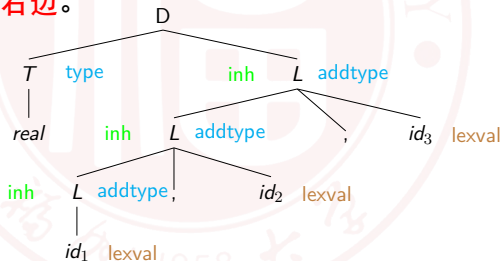
例 (SDD 依赖图示例)

考虑产生式和语义规则如下：

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$ $addtype(id.lexval, L.inh)$
$L \rightarrow id$	$addtype(id.lexval, L.inh)$

输入为 $real\ id_1, id_2, id_3$ 。

为了便于区分，一般将继承属性 (**inh**) 放在分析树结点的**左边**，综合属性 (**type**, **lexeme**) 放在分析树结点的**右边**。



间的**循环依赖关系**（即依赖图中不存在环），则
) 的，依赖图存在拓扑排序。
出一棵语法树中结点的语义规则计算的有效

- 如果一个 SDD 中不存在属性之间的**循环依赖关系**（即依赖图中不存在环），则该文法是**良定义（Well-defined）**的，依赖图存在拓扑排序。
- 一个依赖图的任何拓扑排序都给出一棵语法树中结点的语义规则计算的有效顺序。

属性值的计算正好和自底向上分析建立分析树结

- 对于只包含综合属性的 SDD，属性值的计算正好和自底向上分析建立分析树结点同步进行

属性值的计算正好和自底向上分析建立分析树结
属性的 SDD，不能保证一定存在一个顺序来计算

- 对于只包含综合属性的 SDD，属性值的计算正好和自底向上分析建立分析树结点同步进行
- 对于同时具有继承属性和综合属性的 SDD，不能保证一定存在一个顺序来计算所有的属性值。

属性值的计算正好和自底向上分析建立分析树
属性的 SDD，不能保证一定存在一个顺序来计
算，很难确定是否存在某棵语法分析树，使得
关系，但可以给出一组充分条件，满足这组充

- 对于只包含综合属性的 SDD，属性值的计算正好和自底向上分析建立分析树结点同步进行
- 对于同时具有继承属性和综合属性的 SDD，不能保证一定存在一个顺序来计算所有的属性值。
- 从计算的角度看，给定一个 SDD，很难确定是否存在某棵语法分析树，使得 SDD 的属性之间存在循环依赖关系，但可以给出一组充分条件，满足这组充分条件的 SDD 是无循环依赖的。

- 仅仅使用**综合属性**的 SDD 称为 **S 属性的 SDD**，或 **S-属性定义**、**S-SDD**。

例 (S-SDD 示例)

- 仅仅使用**综合属性**的 SDD 称为 **S 属性的 SDD**，或 **S-属性定义**、**S-SDD**。

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

例 (S-SDD 示例)

- 仅仅使用综合属性的 SDD 称为 **S 属性的 SDD**，或 **S-属性定义**、**S-SDD**。
- 如果一个 SDD 是 S 属性的，可以按照语法分析树节点的任何自底向上顺序来计算它的各个属性值。

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

例 (S-SDD 示例)

- 仅仅使用**综合属性**的 SDD 称为 **S 属性的 SDD**，或 **S-属性定义**、**S-SDD**。
- 如果一个 SDD 是 S 属性的，可以按照语法分析树节点的任何自底向上顺序来计算它的各个属性值。
- S-属性定义可以在自底向上的语法分析过程中实现。

产生式	语义规则
$L \rightarrow E$	$L.val = E.val$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val \times F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow digit$	$F.val = digit.lexval$

OD 或 L-SDD) 的直观含义: 在一个产生式所关
以从左到右, 但不能从右到左 (因此取 Left 的
又当² 它的每个属性要么是一个**综合属性**, 要么
设存在一个产生式 $A \rightarrow X_1 X_2 \dots X_n$, 其右部符
依赖于下列属性:

- 1. $X_1, \dots, X_{i-1}$ 的属性;
- 2. 属性不能在依赖图中形成环路。

属性定义。

- OD 或 L-SDD) 的直观含义: 在一个产生式所关
以从左到右, 但不能从右到左 (因此取 Left 的
又当² 它的每个属性要么是一个**综合属性**, 要么
设存在一个产生式 $A \rightarrow X_1 X_2 \dots X_n$, 其右部符
依赖于下列属性:
- 1. $X_1, \dots, X_{i-1}$ 的属性;
 - 2. 属性不能在依赖图中形成环路。
- 属性定义。

OD 或 L-SDD) 的直观含义: 在一个产生式所关
以从左到右, 但不能从右到左 (因此取 Left 的
又当² 它的每个属性要么是一个**综合属性**, 要么
设存在一个产生式 $A \rightarrow X_1 X_2 \dots X_n$, 其右部符
依赖于下列属性:

- 1. $X_1, \dots, X_{i-1}$ 的属性;
- 2. 属性不能在依赖图中形成环路。

属性定义。

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

L-SDD 示例

例

请指出下列语义规则中哪些是继承属性，哪些是综合属性。

产生式	语义规则
$T \rightarrow FT'$	$T'.inh = F.val$ $T.val = T'.syn$
$T' \rightarrow *FT'$	$T'_1.inh = T.inh \times F.val$ $T'.syn = T'_1.syn$
$T \rightarrow \varepsilon$	$T'.syn = T'.inh$
$F \rightarrow digit$	$F.val = digit.lexval$

例

- | 产生式 | 语义规则 |
|--------------------|--|
| $A \rightarrow LM$ | $L.i = l(A.i)$
$M.i = m(L.s)$
$A.s = f(M.s)$ |
| $A \rightarrow QR$ | $R.i = r(A.i)$
$Q.i = q(R.s)$
$A.s = f(Q.s)$ |

L-SDD 示例 (反例)

例

- ① 请指出下列语义规则中哪些是继承属性，哪些是综合属性。
- ② 请指出哪一条语义规则不是 L-SDD 的。

产生式	语义规则
$A \rightarrow LM$	$L.i = l(A.i)$ $M.i = m(L.s)$ $A.s = f(M.s)$
$A \rightarrow QR$	$R.i = r(A.i)$ $Q.i = q(R.s)$ $A.s = f(Q.s)$

目录

① 中间代码

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

表达式翻译

声明处理

数组访问的翻译

④ 控制流与回填

⑤ 小结与作业



(action) 的上下文无关文法。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

(action) 的上下文无关文法。
「出现在产生式右部的任何位置。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

(action) 的上下文无关文法。
「出现在产生式右部的任何位置。

- SDT 是在产生式体中嵌入动作 (action) 的上下文无关文法。
- 动作用花括号 $\{ \dots \}$ 括起来, 可出现在产生式右部的任何位置。
- 动作的执行时机:

(action) 的上下文无关文法。
出现在产生式右部的任何位置。
右部符号分析完毕后执行（对应综合属性）。

- SDT 是在产生式体中嵌入动作 (action) 的上下文无关文法。
- 动作用花括号 {...} 括起来, 可出现在产生式右部的任何位置。
- 动作的执行时机:
 - 若动作在产生式最右端: 所有右部符号分析完毕后执行 (对应综合属性)。

(action) 的上下文无关文法。
出现在产生式右部的任何位置。
右部符号分析完毕后执行（对应综合属性）。
符号分析完毕、右侧符号尚未分析时执行。

- (action) 的上下文无关文法。
出现在产生式右部的任何位置。
右部符号分析完毕后执行（对应综合属性）。
符号分析完毕、右侧符号尚未分析时执行。

(action) 的上下文无关文法。

可出现在产生式右部的任何位置。

右部符号分析完毕后执行（对应综合属性）。

左部符号分析完毕、右侧符号尚未分析时执行。

转化为递归下降分析器中的代码片段。

- SDT 是在产生式体中嵌入动作 (action) 的上下文无关文法。
- 动作用花括号 {...} 括起来, 可出现在产生式右部的任何位置。
- 动作的执行时机:
 - 若动作在产生式最右端: 所有右部符号分析完毕后执行 (对应综合属性)。
 - 若动作夹在符号中间: 在左侧符号分析完毕、右侧符号尚未分析时执行。
- SDT 更贴近实际实现, 可直接转化为递归下降分析器中的代码片段。

- ① 中间代码
- ② 语法制导定义 (SDD)
- ③ 语法制导翻译 (SDT)
 - 表达式翻译
 - 声明处理
 - 数组访问的翻译
- ④ 控制流与回填
- ⑤ 小结与作业

表达式翻译为后缀式

SDT 方案

$$E \rightarrow E_1 + T \quad \{\text{print}(' + ')\}$$

$$E \rightarrow T$$

$$T \rightarrow T_1 * F \quad \{\text{print}(' * ')\}$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{id} \quad \{\text{print}(\text{id.lexeme})\}$$

表达式翻译为后缀式

SDT 方案

$$E \rightarrow E_1 + T \quad \{\text{print}(' + ')\}$$

$$E \rightarrow T$$

$$T \rightarrow T_1 * F \quad \{\text{print}(' * ')\}$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{id} \quad \{\text{print}(\text{id.lexeme})\}$$

- 按此 SDT 分析 $a + b * c$ 将输出: a b c * +

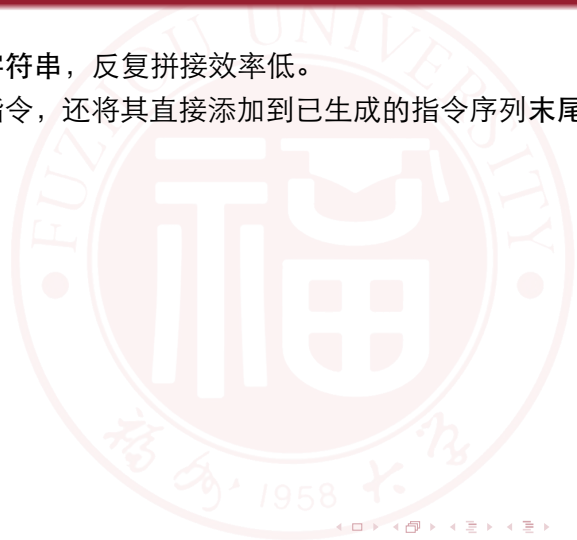
增量翻译

- 上述 SDT 中, *E.code* 是代码字符串, 反复拼接效率低。



增量翻译

- 上述 SDT 中, *E.code* 是代码字符串, 反复拼接效率低。
- 增量翻译: `gen()` 不仅构造新指令, 还将其直接添加到已生成的指令序列末尾。



增量翻译

- 上述 SDT 中, $E.code$ 是代码字符串, 反复拼接效率低。
- 增量翻译: $gen()$ 不仅构造新指令, 还将其直接添加到已生成的指令序列末尾。
- 无需维护 $code$ 属性, 只需维护 $addr$ 属性。

基于增量翻译的 SDT

$S \rightarrow id = E$	$gen(id.lexeme '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = newtemp(); gen(E.addr '=' E_1.addr '+' E_2.addr)$
$E \rightarrow -E_1$	$E.addr = newtemp(); gen(E.addr '=' 'uminus' E_1.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr$
$E \rightarrow id$	$E.addr = id.lexeme$

增量翻译

- 上述 SDT 中, $E.code$ 是代码字符串, 反复拼接效率低。
- 增量翻译: $gen()$ 不仅构造新指令, 还将其直接添加到已生成的指令序列末尾。
- 无需维护 $code$ 属性, 只需维护 $addr$ 属性。
- 这是实际编译器中最常用的实现方式。

基于增量翻译的 SDT

$S \rightarrow id = E$	$gen(id.lexeme '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = newtemp(); gen(E.addr '=' E_1.addr '+' E_2.addr)$
$E \rightarrow -E_1$	$E.addr = newtemp(); gen(E.addr '=' 'uminus' E_1.addr)$
$E \rightarrow (E_1)$	$E.addr = E_1.addr$
$E \rightarrow id$	$E.addr = id.lexeme$

表达式翻译为三地址码

SDD 方案

翻译: $x = a + b * c$

产生式	语义动作
$S \rightarrow id = E$	gen(id.lexeme '=' E.addr)
$E \rightarrow E_1 + E_2$	E.addr = newtemp(); gen(E.addr '=' E ₁ .addr '+' E ₂ .addr)
$E \rightarrow -E_1$	E.addr = newtemp(); gen(E.addr '=' 'uminus' E ₁ .addr)
$E \rightarrow (E_1)$	E.addr = E ₁ .addr
$E \rightarrow id$	E.addr = id.lexeme

表达式翻译为三地址码

SDD 方案

翻译: $x = a + b * c$

- 构造 SDT

产生式	语义动作
$S \rightarrow id = E$	gen(id.lexeme '=' E.addr)
$E \rightarrow E_1 + E_2$	E.addr = newtemp(); gen(E.addr '=' E ₁ .addr '+' E ₂ .addr)
$E \rightarrow -E_1$	E.addr = newtemp(); gen(E.addr '=' 'uminus' E ₁ .addr)
$E \rightarrow (E_1)$	E.addr = E ₁ .addr
$E \rightarrow id$	E.addr = id.lexeme

表达式翻译为三地址码

SDD 方案

产生式	语义动作
$S \rightarrow id = E$	gen(id.lexeme '=' E.addr)
$E \rightarrow E_1 + E_2$	E.addr = newtemp(); gen(E.addr '=' E ₁ .addr '+' E ₂ .addr)
$E \rightarrow -E_1$	E.addr = newtemp(); gen(E.addr '=' 'uminus' E ₁ .addr)
$E \rightarrow (E_1)$	E.addr = E ₁ .addr
$E \rightarrow id$	E.addr = id.lexeme

翻译: $x = a + b * c$

- 构造 SDT
- 画出分析树

表达式翻译为三地址码

SDD 方案

产生式	语义动作
$S \rightarrow id = E$	gen(id.lexeme '=' E.addr)
$E \rightarrow E_1 + E_2$	E.addr = newtemp(); gen(E.addr '=' E ₁ .addr '+' E ₂ .addr)
$E \rightarrow -E_1$	E.addr = newtemp(); gen(E.addr '=' 'uminus' E ₁ .addr)
$E \rightarrow (E_1)$	E.addr = E ₁ .addr
$E \rightarrow id$	E.addr = id.lexeme

翻译: $x = a + b * c$

- 构造 SDT
- 画出分析树
- 输出三地址码

参考答案

t1 = b * c

t2 = a + t1

x = t2

- ① 中间代码
- ② 语法制导定义 (SDD)
- ③ 语法制导翻译 (SDT)
 - 表达式翻译
 - 声明处理
 - 数组访问的翻译
- ④ 控制流与回填
- ⑤ 小结与作业

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。



类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。

类型及存储空间的布局问题。

空间是在运行时进行分配的。在编译时检查布局 (layout)。

相对地址是指它相对于数据区域开始位置 (数据的第一个字节) 的偏移量。

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局（layout）。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - 1 确定一个名需要的存储空间；

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - ① 确定一个名需要的存储空间；
 - ② 计算一个数组元素引用的地址；

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - 1 确定一个名需要的存储空间；
 - 2 计算一个数组元素引用的地址；
 - 3 插入显式的类型转换；

类型与声明

- 在声明变量或过程时，需要考虑**类型**及**存储空间的布局**问题。
- 一个过程调用或对象的实际存储空间是在**运行时**进行分配的。在**编译时**检查局部声明时，可以进行**相对地址**的布局 (layout)。
- 一个名字或某个数据结构分量的**相对地址**是指它相对于数据区域开始位置（如栈顶、数组第一个字节、结构体的第一个字节）的偏移量。
- 类型检查：
 - 定义：利用一组逻辑规则来确定程序在运行时的行为。
 - 目的：保证运算分量的类型和运算符的预期类型相匹配。
 - 翻译时的应用：
 - 1 确定一个名需要的存储空间；
 - 2 计算一个数组元素引用的地址；
 - 3 插入显式的类型转换；
 - 4 选择算术运算符的正确版本；

常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。



常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type)：整数、浮点、字符、布尔、无类型；



常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type): 整数、浮点、字符、布尔、无类型;
- 类型表达式与类型别名:

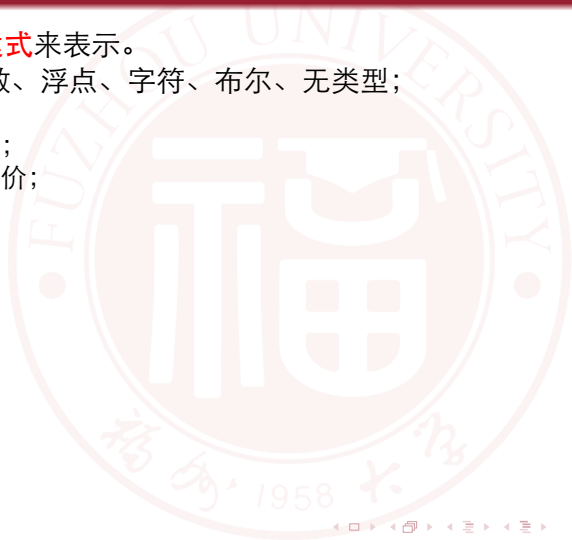


常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type): 整数、浮点、字符、布尔、无类型;
- 类型表达式与类型别名:
 - 类型表达式用于说明一个类型;

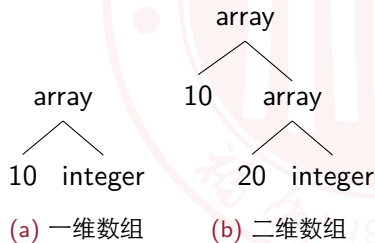
常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type): 整数、浮点、字符、布尔、无类型;
- 类型表达式与类型别名:
 - 类型表达式用于说明一个类型;
 - 类型别名与一串类型表达式等价;



常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type): 整数、浮点、字符、布尔、无类型;
- 类型表达式与类型别名:
 - 类型表达式用于说明一个类型;
 - 类型别名与一串类型表达式等价;
- 数组类型: 一维数组、多维数组; 如 `array(N, T)` 说明了一个有 10 个 T 类型元素构成的数组。



常见类型表达式

- **类型**自身也有**结构**，用**类型表达式**来表示。
- 基本类型 (primitive type): 整数、浮点、字符、布尔、无类型;
- 类型表达式与类型别名:
 - 类型表达式用于说明一个类型;
 - 类型别名与一串类型表达式等价;
- 数组类型: 一维数组、多维数组; 如 `array(N, T)` 说明了一个有 10 个 T 类型元素构成的数组类型。
- 指针类型: 用 `pointer(T)` 表示指向数据类型 T 的指针类型。
- 笛卡尔算符 $\times$: 如果 T_1 与 T_2 是合法的类型表达式, 则 $T_1 \times T_2$ 也是合法的类型表达式。(用在哪儿?)

常见类型表达式

- 函数声明符 $\rightarrow$: 若 T_1 、 T_2 、...、 T_n 和 R 是类型表达式, 则 $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 是合法的类型表达式, 用于表示一个函数。

例

函数 `int *f(char a, char b);` 如何表示?

例

类型表达式: $\text{char} \times \text{char} \rightarrow \text{pointer}(\text{int})$

常见类型表达式

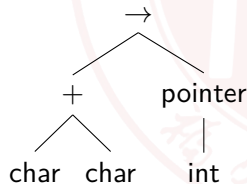
- 函数声明符 $\rightarrow$: 若 T_1 、 T_2 、...、 T_n 和 R 是类型表达式, 则 $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 是合法的类型表达式, 用于表示一个函数。

例

函数 `int *f(char a, char b);` 如何表示?

类型表达式: $\text{char} \times \text{char} \rightarrow \text{pointer}(\text{int})$

对应的树形表示为:



常见类型表达式

- 函数声明符 $\rightarrow$: 若 T_1 、 T_2 、...、 T_n 和 R 是类型表达式, 则 $T_1 \times T_2 \times \cdots \times T_n \rightarrow R$ 是合法的类型表达式, 用于表示一个函数。
- 记录**³ 构造符 record: 若有标识符 N_1 、 N_2 、...、 N_n 与类型表达式 T_1 、 T_2 、...、 T_n , 则 $\text{record}((N_1 \times T_1) \times (N_2 \times T_2) \times \cdots \times (N_n \times T_n))$ 是类型表达式。

例

请写出下列结构体和数组的类型表达式:

```
struct person {
    char name[8];
    int sex;
    int age;
};
```

```
struct person table[50];
```

³结构体、类、命名元组 (namedtuple)

变量声明语句的 SDT

变量声明文法:

- ① $P \rightarrow D$
- ② $D \rightarrow T \text{ id}; D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow B C$
- ⑤ $B \rightarrow \text{int} \mid \text{real}$
- ⑥ $C \rightarrow \varepsilon \mid [\text{num}] C$



变量声明语句的 SDT

变量声明文法:

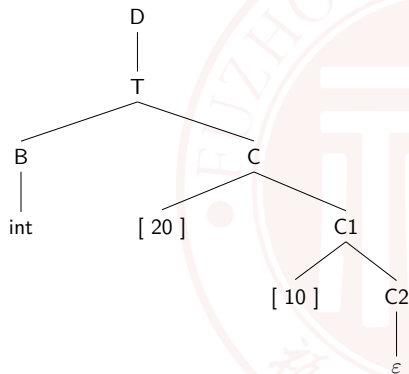
- ① $P \rightarrow D$
- ② $D \rightarrow T \text{ id}; D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow B C$
- ⑤ $B \rightarrow \text{int} \mid \text{real}$
- ⑥ $C \rightarrow \varepsilon \mid [\text{num}] C$

- 翻译要求: 确定每个变量的类型和相对地址。
- 综合属性: $B.type, B.width, T.type, T.width$ 。
- 继承属性/临时变量: t, w 暂存 B 的类型和宽度。
- 辅助变量: $offset$, 下一个可用的相对地址。
- 处理第一个声明前, $offset = 0$ 。

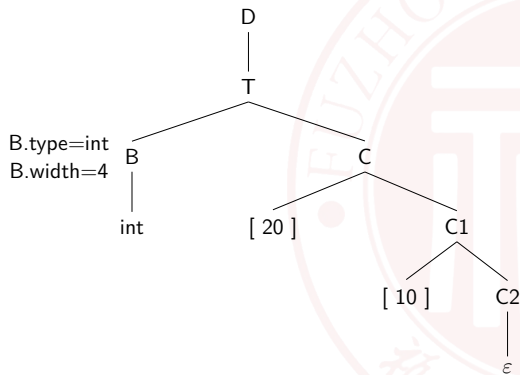
变量声明语句的 SDT

- ① $P \rightarrow \{\text{offset} = 0;\} D$
- ② $D \rightarrow T \text{ id}; \{\text{st.put}(\text{id.lexeme}, T.\text{type}, \text{offset}); \text{offset} = \text{offset} + T.\text{width};\} D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow B \{t = B.\text{type}; w = B.\text{width};\} C \{T.\text{type} = C.\text{type}; T.\text{width} = C.\text{width};\}$
- ⑤ $B \rightarrow \text{int} \{B.\text{type} = \text{int}; B.\text{width} = 4;\}$
- ⑥ $B \rightarrow \text{real} \{B.\text{type} = \text{real}; B.\text{width} = 8;\}$
- ⑦ $C \rightarrow \varepsilon \{C.\text{type} = t; C.\text{width} = w;\}$
- ⑧ $C \rightarrow [\text{num}] C_1 \{C.\text{type} = \text{array}(\text{num.val}, C_1.\text{type}); C.\text{width} = \text{num.val} * C_1.\text{width};\}$

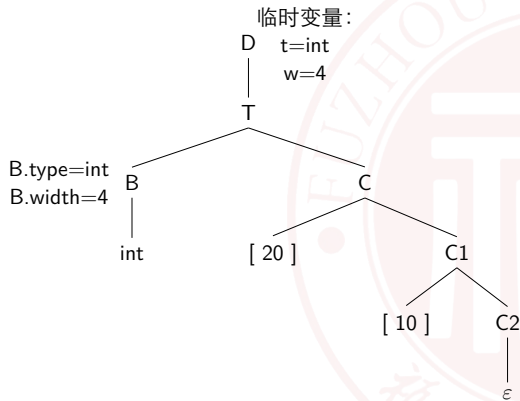
int a[20][10]; 的注释分析树



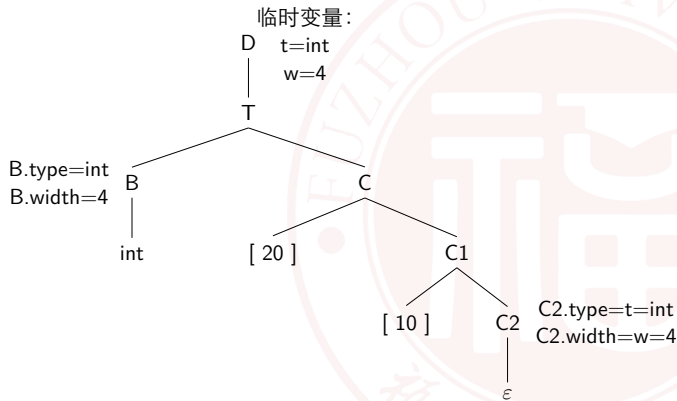
int a[20][10]; 的注释分析树



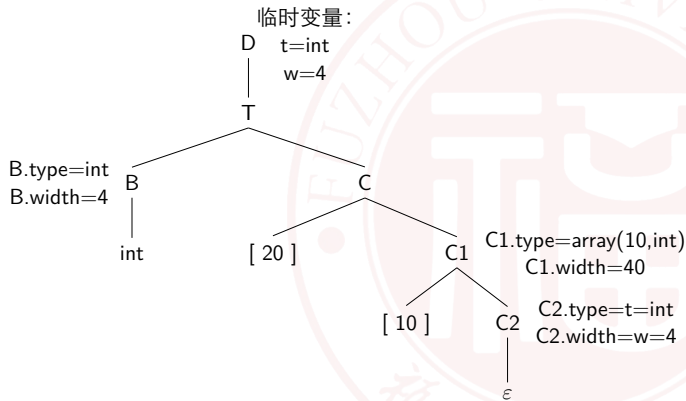
int a[20][10]; 的注释分析树



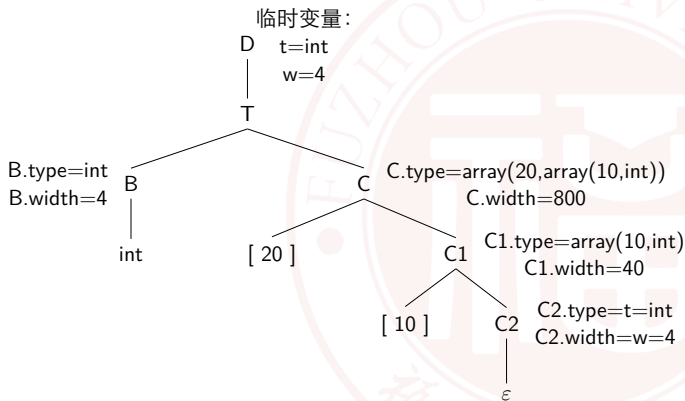
int a[20][10]; 的注释分析树



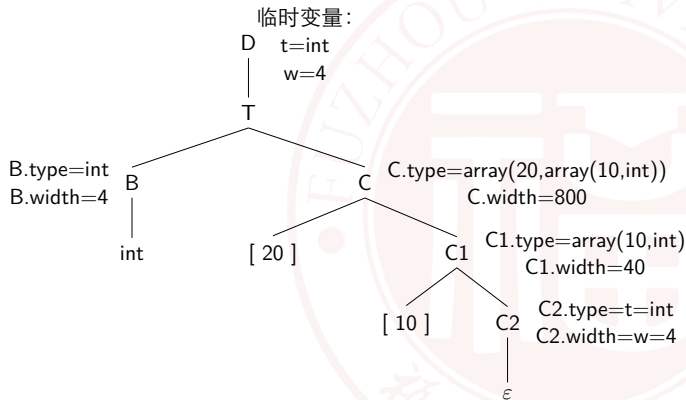
int a[20][10]; 的注释分析树



int a[20][10]; 的注释分析树



int a[20][10]; 的注释分析树



- 关键特征：数组类型从右往左构造（先算内层 [10]，再算外层 [20]）。

器会：

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

器会：
错)；

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ▶ ↺ 🔍 ↻

器会：
错)；

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行自动的类型转换。

器会：
错)；

rt → int → float → double

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行自动的类型转换。
- 拓宽转换（低层 → 高层）：`short` → `int` → `float` → `double`

类型检查与类型转换

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行自动的类型转换。
- 拓宽转换（低层 $\rightarrow$ 高层）：`short  $\rightarrow$  int  $\rightarrow$  float  $\rightarrow$  double`
- 窄化转换（高层 $\rightarrow$ 低层）：可能丢失精度。

类型检查与类型转换

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行自动的类型转换。
- 拓宽转换（低层 $\rightarrow$ 高层）：`short  $\rightarrow$  int  $\rightarrow$  float  $\rightarrow$  double`
- 窄化转换（高层 $\rightarrow$ 低层）：可能丢失精度。
- 隐式转换：由编译器自动完成。

类型检查与类型转换

- 当表达式出现混合运算时，编译器会：
 - 拒绝一些混合类型的运算（报错）；
 - 进行自动的类型转换。
- 拓宽转换（低层 $\rightarrow$ 高层）：`short  $\rightarrow$  int  $\rightarrow$  float  $\rightarrow$  double`
- 窄化转换（高层 $\rightarrow$ 低层）：可能丢失精度。
- 隐式转换：由编译器自动完成。
- 显式转换（强制类型转换）：由程序员写出。

目录

① 中间代码

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

表达式翻译

声明处理

数组访问的翻译

④ 控制流与回填

⑤ 小结与作业



数组元素的地址计算

- 设数组 a 的基地址为 $base$ ，元素类型宽度为 w 。



数组元素的地址计算

- 设数组 a 的基地址为 base ，元素类型宽度为 w 。
- 一维数组： $\text{addr}(a[i]) = \text{base} + i \times w$



数组元素的地址计算

- 设数组 a 的基地址为 $base$ ，元素类型宽度为 w 。
- 一维数组： $addr(a[i]) = base + i \times w$
- 二维数组（行优先）： $addr(a[i][j]) = base + (i \times n_2 + j) \times w$

数组元素的地址计算

- 设数组 a 的基地址为 base ，元素类型宽度为 w 。
- 一维数组： $\text{addr}(a[i]) = \text{base} + i \times w$
- 二维数组（行优先）： $\text{addr}(a[i][j]) = \text{base} + (i \times n_2 + j) \times w$
- k 维行优先通用公式：

$$\text{addr} = \text{base} + ((\dots((i_1 \times n_2 + i_2) \times n_3 + i_3) \dots) \times n_k + i_k) \times w$$

数组内存布局：行优先 vs 列优先

a[2][3] 按行存放
(C/C++/Java)

——	a[1][2]
第二行	a[1][1]
——	a[1][0]
——	a[0][2]
第一行	a[0][1]
——	a[0][0]



数组内存布局：行优先 vs 列优先

a[2][3] 按行存放
(C/C++/Java)

	a[1][2]
第二行	a[1][1]
	a[1][0]
	a[0][2]
第一行	a[0][1]
	a[0][0]

a[2][3] 按列存放
(Fortran/MATLAB)

	a[1][2]
第三列	a[0][2]
	a[1][1]
第二列	a[0][1]
	a[1][0]
第一列	a[0][0]

数组引用寻址的 SDT

- 关键：将地址计算公式同数组引用的文法关联起来。



用的文法关联起来。

表达式的序列：

$$L \rightarrow \text{id}[E] \mid L_1[E]$$

- $$L \rightarrow \text{id}[E] \mid L_1[E]$$

数组引用寻址的 SDT

- 关键：将地址计算公式同数组引用的文法关联起来。
- 非终结符 L 生成标识符连接下标表达式的序列：

$$L \rightarrow \text{id}[E] \mid L_1[E]$$

- L 的综合属性：

数组引用寻址的 SDT

- 关键：将地址计算公式同数组引用的文法关联起来。
- 非终结符 L 生成标识符连接下标表达式的序列：

$$L \rightarrow \text{id}[E] \mid L_1[E]$$

- L 的综合属性：
 - $L.\text{addr}$ ：临时变量，表示偏移量 $\sum_j i_j \times w_j$ 。

数组引用寻址的 SDT

- 关键：将地址计算公式同数组引用的文法关联起来。
- 非终结符 L 生成标识符连接下标表达式的序列：

$$L \rightarrow \text{id}[E] \mid L_1[E]$$

- L 的综合属性：
 - $L.\text{addr}$ ：临时变量，表示偏移量 $\sum_j i_j \times w_j$ 。
 - $L.\text{array}$ ：指向数组名在符号表中的条目。

数组引用寻址的 SDT

- 关键：将地址计算公式同数组引用的文法关联起来。
- 非终结符 L 生成标识符连接下标表达式的序列：

$$L \rightarrow \text{id}[E] \mid L_1[E]$$

- L 的综合属性：
 - $L.\text{addr}$ ：临时变量，表示偏移量 $\sum_j i_j \times w_j$ 。
 - $L.\text{array}$ ：指向数组名在符号表中的条目。
 - $L.\text{type}$ ： L 生成的子数组的类型，用于获取元素宽度。

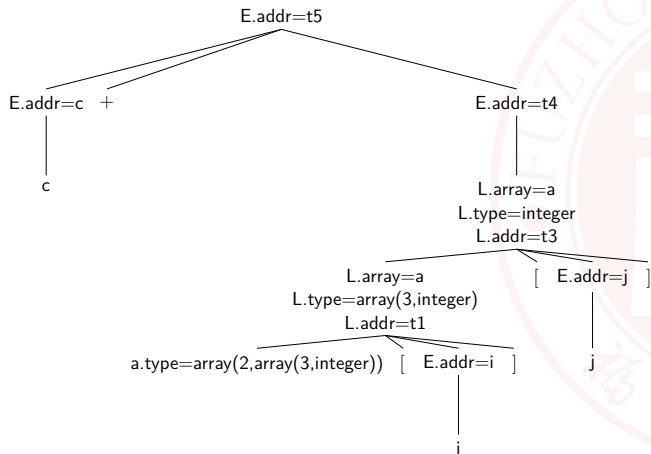
数组引用寻址的 SDT (续)

$S \rightarrow id = E;$	{gen(top.get(id.lexeme), '=', E.addr);}
$L = E;$	{gen(L.array.base, '[', L.addr, ']', '=', E.addr);}
$E \rightarrow E_1 + E_2$	{E.addr=newTemp(); gen(E.addr, '=', E1.addr, '+', E2.addr);}
id	{E.addr=top.get(id.lexeme);}
L	{E.addr=newTemp(); gen(E.addr, '=', L.array.base, '[', L.addr, ']');}
$L \rightarrow id[E]$	{L.array=top.get(id.lexeme); L.type=L.array.type.elem;
	L.addr=newTemp(); gen(L.addr, '=', E.addr, '*', L.type.width);}
$L_1[E]$	{L.array=L1.array; L.type=L1.type.elem; t=newTemp();
	L.addr=newTemp(); gen(t, '=', E.addr, '*', L.type.width);
	gen(L.addr, '=', L1.addr, '+', t);}

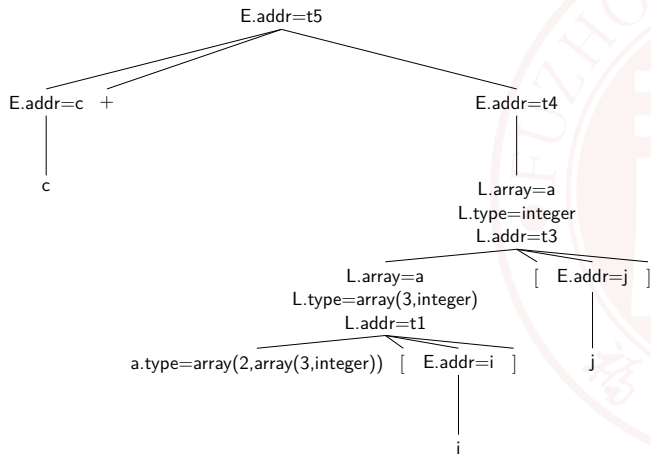
$S \rightarrow id = E;$	{gen(top.get(id.lexeme), '=', E.addr);}
$ L = E;$	{gen(L.array.base, '[', L.addr, ']', '=', E.addr);}
$E \rightarrow E_1 + E_2$	{E.addr=newTemp(); gen(E.addr, '=', E1.addr, '+', E2.addr);}
$ id$	{E.addr=top.get(id.lexeme);}
$ L$	{E.addr=newTemp(); gen(E.addr, '=', L.array.base, '[', L.addr, ']');}
$L \rightarrow id[E]$	{L.array=top.get(id.lexeme); L.type=L.array.type.elem;
	L.addr=newTemp(); gen(L.addr, '=', E.addr, '*', L.type.width);}
$ L_1[E]$	{L.array=L1.array; L.type=L1.type.elem; t=newTemp();
	L.addr=newTemp(); gen(t, '=', E.addr, '*', L.type.width);
	gen(L.addr, '=', L1.addr, '+', t);}

福州大学计算机与大数据学院

int a[2][3]; c + a[i][j]; 的注释分析树



`int a[2][3]; c = a[i][j];` 的注释分析树



生成的三地址码

`t1 = i * 12`

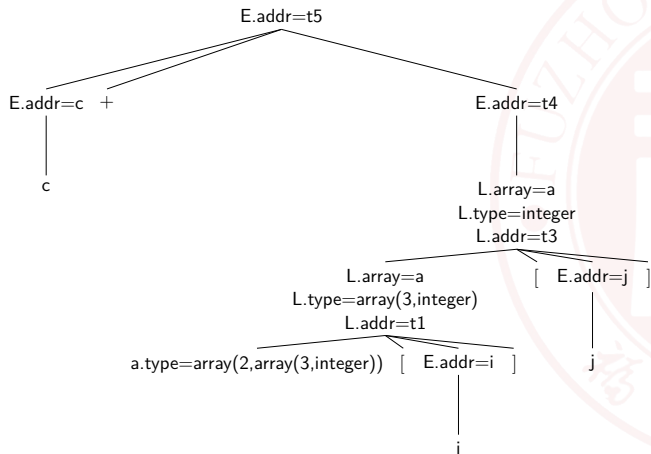
`t2 = j * 4`

`t3 = t1 + t2`

`t4 = a[t3]`

`t5 = c + t4`

`int a[2][3]; c + a[i][j];` 的注释分析树



生成的三地址码

`t1 = i * 12`

`t2 = j * 4`

`t3 = t1 + t2`

`t4 = a[t3]`

`t5 = c + t4`

要点总结

- 数组声明**从右往左**构造类型
- 数组寻址**从左往右**剥离维度

目录

① 中间代码

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

④ 控制流与回填

控制流语句的代码结构

回填技术

switch 语句的翻译

⑤ 小结与作业

布尔表达式的计算方法

数值法

- 用 1 表示真，0 表示假。
- 像算术表达式一样逐步计算。

例

$$\begin{aligned} & 1 || (!0 \& \& 0) || 0 \\ &= 1 || (1 \& \& 0) || 0 \\ &= 1 || 0 || 0 = 1 \end{aligned}$$

布尔表达式的计算方法

数值法

- 用 1 表示真，0 表示假。
- 像算术表达式一样逐步计算。

例

$$\begin{aligned}
 & 1 || (!0 \& \& 0) || 0 \\
 & = 1 || (1 \& \& 0) || 0 \\
 & = 1 || 0 || 0 = 1
 \end{aligned}$$

短路跳转法

- $A || B$: 若 A 为真，则不必计算 B。
- $A \& \& B$: 若 A 为假，则不必计算 B。
- 运算符被翻译成跳转指令，不出现在代码中。

例

```
if (x < 100 || x > 200 && x != y) x = 0;
```

短路代码示例

if (x < 100 || x > 200 && x != y) x = 0; 的三地址码

```

        if x<100 goto L2
        goto L3
L3:     if x>200 goto L4
        goto L1
L4:     if x!=y goto L2
        goto L1
L2:     x=0
L1:

```


目录

① 中间代码

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

④ 控制流与回填

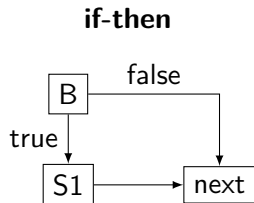
控制流语句的代码结构

回填技术

switch 语句的翻译

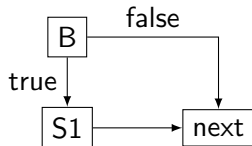
⑤ 小结与作业

控制流语句的结构

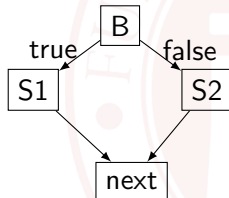


控制流语句的结构

if-then

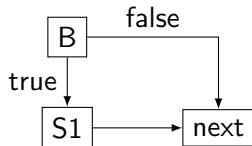


if-then-else

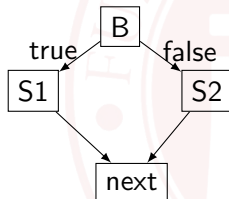


控制流语句的结构

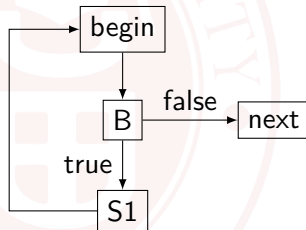
if-then



if-then-else



while-do



控制流语句的 SDT (非回填版)

- 关键：用指令的标号标识一条三地址指令。



地址指令。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

地址指令。

表号。

A large, faint watermark of Fuzhou University's logo is centered in the background. It features a circular seal with the university's name in English, 'FUZHOU UNIVERSITY', at the top and '福州大学' in Chinese at the bottom, with the year '1958' in the middle. In the center of the seal is a large white character '福' (Fú) with a graduation cap on top.

- 地址指令。
- 表号。
- 
- A large, faint watermark of Fuzhou University's logo is centered in the background. It features a circular seal with the university's name in English, 'FUZHOU UNIVERSITY', at the top and '福州大学' in Chinese at the bottom, with the year '1958' in the middle. In the center of the seal is a large white character '福' (Fú) with a graduation cap on top.

三地址指令。

标号。

一个生成的三地址码。

- 关键：用指令的标号标识一条三地址指令。
- 引入辅助函数：
 - `newlabel()`：产生一个新的标号。
 - `label(L)`：将标签 `L` 赋予下一个生成的三地址码。

控制流语句的 SDT (非回填版)

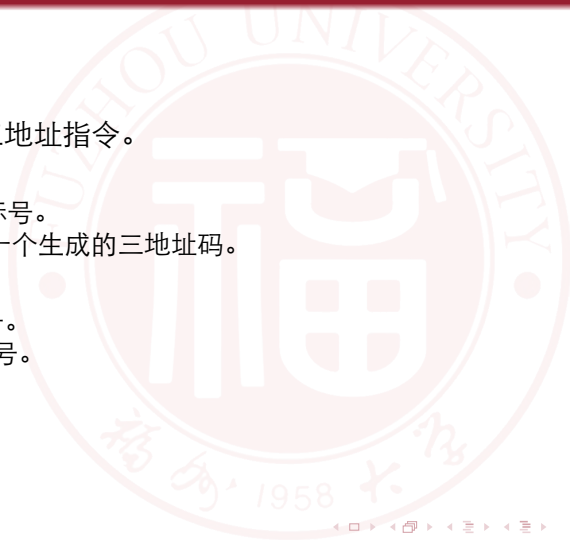
- 关键：用指令的标号标识一条三地址指令。
- 引入辅助函数：
 - newlabel(): 产生一个新的标号。
 - label(L): 将标签 L 赋予下一个生成的三地址码。
- 继承属性：

控制流语句的 SDT (非回填版)

- 关键：用指令的标号标识一条三地址指令。
- 引入辅助函数：
 - newlabel(): 产生一个新的标号。
 - label(L): 将标签 L 赋予下一个生成的三地址码。
- 继承属性：
 - B.true: B 为真时的跳转标号。

控制流语句的 SDT (非回填版)

- 关键：用指令的标号标识一条三地址指令。
- 引入辅助函数：
 - newlabel(): 产生一个新的标号。
 - label(L): 将标签 L 赋予下一个生成的三地址码。
- 继承属性：
 - B.true: B 为真时的跳转标号。
 - B.false: B 为假时的跳转标号。



控制流语句的 SDT (非回填版)

- 关键：用指令的标号标识一条三地址指令。
- 引入辅助函数：
 - newlabel(): 产生一个新的标号。
 - label(L): 将标签 L 赋予下一个生成的三地址码。
- 继承属性：
 - B.true: B 为真时的跳转标号。
 - B.false: B 为假时的跳转标号。
 - S.next: 紧跟在 S 代码之后的指令标号。

分支与循环的 SDT

if (B) S1 的翻译方案:

$$S \rightarrow \text{if} (\{B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next};\} B \\ \{ \text{label}(B.\text{true}); S1.\text{next} = S.\text{next};\}) S1$$

分支与循环的 SDT

if (B) S1 的翻译方案:

$S \rightarrow \text{if} (\{B.\text{true} = \text{newlabel}(); B.\text{false} = S.\text{next};\} B$
 $\{ \text{label}(B.\text{true}); S1.\text{next} = S.\text{next}; \}) S1$

if (B) S1 else S2 的翻译方案:

$S \rightarrow \text{if} (\{B.\text{true} = \text{newlabel}(); B.\text{false} = \text{newlabel}();\} B$
 $\{ \text{label}(B.\text{true}); S1.\text{next} = S.\text{next}; \}) S1$
 $\{ \text{gen}('goto', S.\text{next}) \} \text{else} \{ \text{label}(B.\text{false}); S2.\text{next} = S.\text{next}; \} S2$

分支与循环的 SDT

if (B) S1 的翻译方案:

```
S → if ( {B.true = newlabel(); B.false = S.next;} B
        {label(B.true); S1.next = S.next;} ) S1
```

if (B) S1 else S2 的翻译方案:

```
S → if ( {B.true = newlabel(); B.false = newlabel();} B
        {label(B.true); S1.next = S.next;} ) S1
        {gen('goto', S.next)} else {label(B.false); S2.next = S.next;} S2
```

while (B) S1 的翻译方案:

```
S → while ( {S.begin = newlabel(); label(S.begin);
            B.true = newlabel(); B.false = S.next;} B
            {label(B.true); S1.next = S.begin;} ) S1 {gen('goto', S.begin);}
```

示例

写出 while (a<b) if (c<d) x=y+z; else x=y-z; 的三地址码

L0: if a<b goto L1
 goto Lnext

L1: if c<d goto L2
 goto L3

L2: x = y + z
 goto L0

L3: x = y - z
 goto L0

Lnext:

示例

写出 while (a<b) if (c<d) x=y+z; else x=y-z; 的三地址码

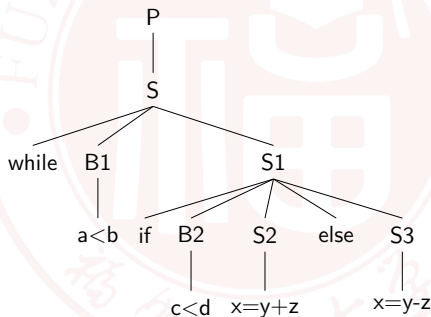
L0: if a<b goto L1
goto Lnext

L1: if c<d goto L2
goto L3

L2: x = y + z
goto L0

L3: x = y - z
goto L0

Lnext:



目录

① 中间代码

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

④ 控制流与回填

控制流语句的代码结构

回填技术

switch 语句的翻译

⑤ 小结与作业

跳转指令的目标位置。

- ⁵LaTeX 在编译 PDF 是就是这么处理交叉引用的

为什么需要回填?

- 控制流语句翻译的关键是确定跳转指令的目标位置。
- **问题**: 生成跳转指令时, 目标位置还不能确定⁴

⁴对应的指令尚未生成, 没有相对位置/绝对位置)。

⁵LaTeX 在编译 PDF 是就是这么处理交叉引用的

为什么需要回填?

- 控制流语句翻译的关键是确定跳转指令的目标位置。
- **问题**: 生成跳转指令时, 目标位置还不能确定⁴
- 解决方案:

⁴对应的指令尚未生成, 没有相对位置/绝对位置)。

⁵LaTeX 在编译 PDF 是就是这么处理交叉引用的

为什么需要回填?

- 控制流语句翻译的关键是确定跳转指令的目标位置。
- **问题**: 生成跳转指令时, 目标位置还不能确定⁴
- 解决方案:
 - ① 两遍扫描: 先生成带临时标号的代码, 第二遍再回填⁵。

⁴对应的指令尚未生成, 没有相对位置/绝对位置)。

⁵LaTeX 在编译 PDF 是就是这么处理交叉引用的

为什么需要回填?

- 控制流语句翻译的关键是确定跳转指令的目标位置。
- **问题**: 生成跳转指令时, 目标位置还不能确定⁴
- **解决方案**:
 - ① 两遍扫描: 先生成带临时标号的代码, 第二遍再回填⁵。
 - ② 一遍扫描 + 回填: 生成跳转指令时暂不指定目标, 将同目标的指令放入一个列表, 待目标确定后统一回填。

⁴对应的指令尚未生成, 没有相对位置/绝对位置)。

⁵LaTeX 在编译 PDF 是就是这么处理交叉引用的

为什么需要回填?

- 控制流语句翻译的关键是确定跳转指令的目标位置。
- **问题**: 生成跳转指令时, 目标位置还不能确定⁴
- **解决方案**:
 - ① 两遍扫描: 先生成带临时标号的代码, 第二遍再回填⁵。
 - ② 一遍扫描 + 回填: 生成跳转指令时暂不指定目标, 将同目标的指令放入一个列表, 待目标确定后统一回填。
- 现代编译系统通常输出成带 Label 的汇编语句, 具体回填由汇编器完成。

⁴对应的指令尚未生成, 没有相对位置/绝对位置)。

⁵LaTeX 在编译 PDF 是就是这么处理交叉引用的

回填的基本思想

- 每个布尔表达式 B 关联两个属性:



回填的基本思想

- 每个布尔表达式 B 关联两个属性：
 - $B.truelist$: 当 B 为真时需要回填的跳转指令列表。



性：

回填的跳转指令列表。

回填的跳转指令列表。

- 每个布尔表达式 B 关联两个属性：
 - $B.truelist$: 当 B 为真时需要回填的跳转指令列表。
 - $B.falselist$: 当 B 为假时需要回填的跳转指令列表。

回填的基本思想

- 每个布尔表达式 B 关联两个属性：
 - $B.truelist$: 当 B 为真时需要回填的跳转指令列表。
 - $B.falselist$: 当 B 为假时需要回填的跳转指令列表。
- 辅助函数:



回填的基本思想

- 每个布尔表达式 B 关联两个属性：
 - $B.truelist$: 当 B 为真时需要回填的跳转指令列表。
 - $B.falselist$: 当 B 为假时需要回填的跳转指令列表。
- 辅助函数：
 - $makelist(i)$: 创建只含索引 i 的列表。

回填的基本思想

- 每个布尔表达式 B 关联两个属性：
 - $B.\text{truelist}$: 当 B 为真时需要回填的跳转指令列表。
 - $B.\text{falselist}$: 当 B 为假时需要回填的跳转指令列表。
- 辅助函数：
 - $\text{makelist}(i)$: 创建只含索引 i 的列表。
 - $\text{merge}(p_1, p_2)$: 合并两个列表。

回填的基本思想

- 每个布尔表达式 B 关联两个属性：
 - $B.truelist$: 当 B 为真时需要回填的跳转指令列表。
 - $B.falselist$: 当 B 为假时需要回填的跳转指令列表。
- 辅助函数：
 - $makelist(i)$: 创建只含索引 i 的列表。
 - $merge(p_1, p_2)$: 合并两个列表。
 - $backpatch(p, i)$: 将列表 p 中所有指令的目标地址回填为 i 。

回填的基本思想

- 每个布尔表达式 B 关联两个属性：
 - $B.\text{truelist}$: 当 B 为真时需要回填的跳转指令列表。
 - $B.\text{falselist}$: 当 B 为假时需要回填的跳转指令列表。
- 辅助函数：
 - $\text{makelist}(i)$: 创建只含索引 i 的列表。
 - $\text{merge}(p_1, p_2)$: 合并两个列表。
 - $\text{backpatch}(p, i)$: 将列表 p 中所有指令的目标地址回填为 i 。
 - $\text{emit}(\dots)$: 生成一条三地址指令，并返回其索引。

布尔表达式的 SDT (回填)

产生式	语义动作
$B \rightarrow B_1 B_2$	$\text{backpatch}(B_1.\text{falselist}, B_2.\text{instr});$ $B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist});$ $B.\text{falselist} = B_2.\text{falselist}$
$B \rightarrow B_1 \&\& B_2$	$\text{backpatch}(B_1.\text{truelist}, B_2.\text{instr});$ $B.\text{truelist} = B_2.\text{truelist};$ $B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist})$
$B \rightarrow ! B_1$	$B.\text{truelist} = B_1.\text{falselist};$ $B.\text{falselist} = B_1.\text{truelist}$
$B \rightarrow (B_1)$	$B.\text{truelist} = B_1.\text{truelist};$ $B.\text{falselist} = B_1.\text{falselist}$
$B \rightarrow E_1 \text{ relop } E_2$	$B.\text{truelist} = \text{makelist}(\text{nextinstr});$ $B.\text{falselist} = \text{makelist}(\text{nextinstr} + 1);$ $\text{emit}(' \text{if } E_1.\text{addr relop } E_2.\text{addr } \text{'goto } _ \text{'});$ $\text{emit}(' \text{goto } _ \text{'})$

示例: $a < b \parallel c < d \&\& e < f$

分析过程



示例: $a < b \parallel c < d \&\& e < f$

分析过程

- ① $a < b$: 生成
 - (0) if $a < b$ goto ___
 - (1) goto ___ $B_1.\text{truelist} = \{0\}, B_1.\text{falselist} = \{1\}$



示例: $a < b \parallel c < d \&\& e < f$

分析过程

- ① $a < b$: 生成
 - (0) if $a < b$ goto ___
 - (1) goto ___
 - $B_1.\text{truelist} = \{0\}, B_1.\text{falselist} = \{1\}$
- ② $c < d$: 生成
 - (2) if $c < d$ goto ___
 - (3) goto ___
 - $B_2.\text{truelist} = \{2\}, B_2.\text{falselist} = \{3\}$



示例: $a < b \parallel c < d \&\& e < f$

分析过程

- ① $a < b$: 生成
 - (0) if $a < b$ goto ___
 - (1) goto ___
 - $B_1.\text{truelist} = \{0\}, B_1.\text{falselist} = \{1\}$
- ② $c < d$: 生成
 - (2) if $c < d$ goto ___
 - (3) goto ___
 - $B_2.\text{truelist} = \{2\}, B_2.\text{falselist} = \{3\}$
- ③ $e < f$: 生成
 - (4) if $e < f$ goto ___
 - (5) goto ___
 - $B_3.\text{truelist} = \{4\}, B_3.\text{falselist} = \{5\}$



示例: $a < b \parallel c < d \&\& e < f$

分析过程

- ① $a < b$: 生成
 - (0) if $a < b$ goto ___
 - (1) goto ___
 - $B_1.truelist = \{0\}, B_1.falselist = \{1\}$
- ② $c < d$: 生成
 - (2) if $c < d$ goto ___
 - (3) goto ___
 - $B_2.truelist = \{2\}, B_2.falselist = \{3\}$
- ③ $e < f$: 生成
 - (4) if $e < f$ goto ___
 - (5) goto ___
 - $B_3.truelist = \{4\}, B_3.falselist = \{5\}$

回填过程

- ① $B_4 = B_2 \&\& B_3$:
 - backpatch($B_2.truelist, 4$)
 - $B_4.truelist = \{4\}$
 - $B_4.falselist = \{3, 5\}$

示例: $a < b \parallel c < d \&\& e < f$

分析过程

- ① $a < b$: 生成
 - (0) if $a < b$ goto ___
 - (1) goto ___
 - $B_1.truelist = \{0\}, B_1.falselist = \{1\}$
- ② $c < d$: 生成
 - (2) if $c < d$ goto ___
 - (3) goto ___
 - $B_2.truelist = \{2\}, B_2.falselist = \{3\}$
- ③ $e < f$: 生成
 - (4) if $e < f$ goto ___
 - (5) goto ___
 - $B_3.truelist = \{4\}, B_3.falselist = \{5\}$

回填过程

- ① $B_4 = B_2 \&\& B_3$:
 - backpatch($B_2.truelist, 4$)
 - $B_4.truelist = \{4\}$
 - $B_4.falselist = \{3, 5\}$
- ② $B = B_1 \parallel B_4$:
 - backpatch($B_1.falselist, 2$)
 - $B.truelist = \{0, 4\}$
 - $B.falselist = \{3, 5\}$

示例: $a < b \parallel c < d \&\& e < f$

分析过程

- ① $a < b$: 生成
 - (0) if $a < b$ goto ___
 - (1) goto ___
 - $B_1.\text{truelist} = \{0\}, B_1.\text{falselist} = \{1\}$
- ② $c < d$: 生成
 - (2) if $c < d$ goto ___
 - (3) goto ___
 - $B_2.\text{truelist} = \{2\}, B_2.\text{falselist} = \{3\}$
- ③ $e < f$: 生成
 - (4) if $e < f$ goto ___
 - (5) goto ___
 - $B_3.\text{truelist} = \{4\}, B_3.\text{falselist} = \{5\}$

回填过程

- ① $B_4 = B_2 \&\& B_3$:
 - backpatch($B_2.\text{truelist}, 4$)
 - $B_4.\text{truelist} = \{4\}$
 - $B_4.\text{falselist} = \{3, 5\}$
- ② $B = B_1 \parallel B_4$:
 - backpatch($B_1.\text{falselist}, 2$)
 - $B.\text{truelist} = \{0, 4\}$
 - $B.\text{falselist} = \{3, 5\}$
- ③ 若真出口在指令 100:
 - backpatch($\{0, 4\}, 100$)

示例: $a < b \parallel c < d \&\& e < f$

分析过程

- ① $a < b$: 生成
 - (0) if $a < b$ goto ___
 - (1) goto ___
 - $B_1.truelist = \{0\}, B_1.falselist = \{1\}$
- ② $c < d$: 生成
 - (2) if $c < d$ goto ___
 - (3) goto ___
 - $B_2.truelist = \{2\}, B_2.falselist = \{3\}$
- ③ $e < f$: 生成
 - (4) if $e < f$ goto ___
 - (5) goto ___
 - $B_3.truelist = \{4\}, B_3.falselist = \{5\}$

回填过程

- ① $B_4 = B_2 \&\& B_3$:
 - backpatch($B_2.truelist, 4$)
 - $B_4.truelist = \{4\}$
 - $B_4.falselist = \{3, 5\}$
- ② $B = B_1 \parallel B_4$:
 - backpatch($B_1.falselist, 2$)
 - $B.truelist = \{0, 4\}$
 - $B.falselist = \{3, 5\}$
- ③ 若真出口在指令 100:
 - backpatch($\{0, 4\}, 100$)
- ④ 若假出口在指令 200:
 - backpatch($\{3, 5\}, 200$)

$a < b \parallel c < d \&\& e < f$ 的回填结果

最终三地址码

```
(0)  if a < b goto 100
(1)  goto 2
(2)  if c < d goto 4
(3)  goto 200
(4)  if e < f goto 100
(5)  goto 200
```

$a < b \parallel c < d \&\& e < f$ 的回填结果

最终三地址码

```
(0)  if a < b goto 100
(1)  goto 2
(2)  if c < d goto 4
(3)  goto 200
(4)  if e < f goto 100
(5)  goto 200
```

分析

- 指令 (1) 被回填为 2, 因为 B_1 为假时需继续判断 B_4 。
- 指令 (0) 和 (4) 被回填为 100 (真出口)。
- 指令 (3) 和 (5) 被回填为 200 (假出口)。

控制流语句的回填 SDT

产生式	语义动作
$S \rightarrow \text{if } (B) S_1$	$\text{backpatch}(B.\text{truelist}, S_1.\text{instr});$ $S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist})$
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$\text{backpatch}(B.\text{truelist}, S_1.\text{instr});$ $\text{backpatch}(B.\text{falselist}, S_2.\text{instr});$ $S.\text{nextlist} = \text{merge}(S_1.\text{nextlist}, S_2.\text{nextlist})$
$S \rightarrow \text{while } M_1 (B) S_1$	$\text{backpatch}(S_1.\text{nextlist}, M_1.\text{instr});$ $\text{backpatch}(B.\text{truelist}, S_1.\text{instr});$ $S.\text{nextlist} = B.\text{falselist};$ $\text{emit}(' \text{goto } M_1.\text{instr}')$

目录

① 中间代码

② 语法制导定义 (SDD)

③ 语法制导翻译 (SDT)

④ 控制流与回填

控制流语句的代码结构

回填技术

switch 语句的翻译

⑤ 小结与作业

case 标签的分布:

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ◀ ↺ 🔍 ↻

switch-case 语句的翻译

- switch-case 的翻译策略取决于 case 标签的分布：
 - 跳转表 (Jump Table):
 - 适用场景: 标签密集且连续 (如 1,2,3,4)
 - 复杂度: $O(1)$



switch-case 语句的翻译

- switch-case 的翻译策略取决于 case 标签的分布：
 - 跳转表 (Jump Table):
 - 适用场景: 标签密集且连续 (如 1,2,3,4)
 - 复杂度: $O(1)$
 - 条件分支链
 - 适用场景: 标签稀疏 (如 10,20,30)
 - 复杂度: $O(n)$

switch-case 语句的翻译

- switch-case 的翻译策略取决于 case 标签的分布：
 - 跳转表 (Jump Table):
 - 适用场景: 标签密集且连续 (如 1,2,3,4)
 - 复杂度: $O(1)$
 - 条件分支链
 - 适用场景: 标签稀疏 (如 10,20,30)
 - 复杂度: $O(n)$
 - 二分查找优化
 - 适用场景: case 数量多且范围大 (如 100,200,...,1000)
 - 复杂度: $O(\log n)$

switch-case 语句的翻译

- switch-case 的翻译策略取决于 case 标签的分布：
 - 跳转表 (Jump Table):
 - 适用场景: 标签密集且连续 (如 1,2,3,4)
 - 复杂度: $O(1)$
 - 条件分支链
 - 适用场景: 标签稀疏 (如 10,20,30)
 - 复杂度: $O(n)$
 - 二分查找优化
 - 适用场景: case 数量多且范围大 (如 100,200,...,1000)
 - 复杂度: $O(\log n)$
- 特别需要注意:
 - **Default 处理**: 所有未匹配的 case 必须跳转到 default 分支。
 - **Fall-through 消除**: break 被翻译为显式的 goto。
 - **值范围检查**: 跳转表需先验证输入值是否在合法范围内。

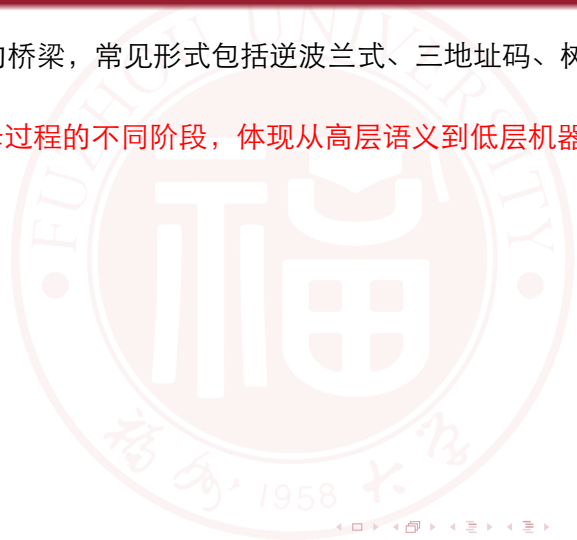
The logo of Fuzhou University is a large, faint watermark in the background. It is a circular seal with the text 'FUZHOU UNIVERSITY' around the top and '福州 1958' (Fuzhou 1958) around the bottom. In the center is a stylized white character '福' (Fú).

- ① 中间代码
- ② 语法制导定义 (SDD)
- ③ 语法制导翻译 (SDT)
- ④ 控制流与回填
- ⑤ 小结与作业
本讲小结
作业

- 桥梁，常见形式包括逆波兰式、三地址码、树

本讲小结

- 中间代码是编译器前端与后端的桥梁，常见形式包括逆波兰式、三地址码、树形 IR 和 LLVM IR。
- **HIR/MIR/LIR** 分别服务于编译过程的不同阶段，体现从高层语义到低层机器的逐步降维。



桥梁，常见形式包括逆波兰式、三地址码、树状结构等。编译过程的不同阶段，体现从高层语义到低层机器语言实现的逐步转化。静态语义与语义规则将语义计算与语法分析结合；声部输出模块负责生成目标代码。

福州大学

- 中间代码是编译器前端与后端的桥梁，常见形式包括逆波兰式、三地址码、树形 IR 和 LLVM IR。
- **HIR/MIR/LIR** 分别服务于编译过程的不同阶段，体现从高层语义到低层机器的逐步降维。
- 语法制导定义（**SDD**）通过属性与语义规则将语义计算与语法分析结合；声明处理需关注类型表达式与存储布局。

- 中间代码是编译器前端与后端的桥梁，常见形式包括逆波兰式、三地址码、树形 IR 和 LLVM IR。
- **HIR/MIR/LIR** 分别服务于编译过程的不同阶段，体现从高层语义到低层机器的逐步降维。
- 语法制导定义（**SDD**）通过属性与语义规则将语义计算与语法分析结合；声明处理需关注类型表达式与存储布局。
- 语法制导翻译（**SDT**）将语义动作嵌入产生式，增量翻译避免了冗长的字符串拼接。

本讲小结

- 中间代码是编译器前端与后端的桥梁，常见形式包括逆波兰式、三地址码、树形 IR 和 LLVM IR。
- **HIR/MIR/LIR** 分别服务于编译过程的不同阶段，体现从高层语义到低层机器的逐步降维。
- 语法制导定义（**SDD**）通过属性与语义规则将语义计算与语法分析结合；声明处理需关注类型表达式与存储布局。
- 语法制导翻译（**SDT**）将语义动作嵌入产生式，增量翻译避免了冗长的字符串拼接。
- 数组翻译的核心是地址计算，声明从右往左构造类型，寻址从左往右剥离维度。

本讲小结

- 中间代码是编译器前端与后端的桥梁，常见形式包括逆波兰式、三地址码、树形 IR 和 LLVM IR。
- **HIR/MIR/LIR** 分别服务于编译过程的不同阶段，体现从高层语义到低层机器的逐步降维。
- 语法制导定义 (**SDD**) 通过属性与语义规则将语义计算与语法分析结合；声明处理需关注类型表达式与存储布局。
- 语法制导翻译 (**SDT**) 将语义动作嵌入产生式，增量翻译避免了冗长的字符串拼接。
- 数组翻译的核心是地址计算，声明从右往左构造类型，寻址从左往右剥离维度。
- 控制流翻译有数值法和短路跳转法；回填技术解决了一遍扫描中跳转目标不确定的问题。

本讲小结

- 中间代码是编译器前端与后端的桥梁，常见形式包括逆波兰式、三地址码、树形 IR 和 LLVM IR。
- **HIR/MIR/LIR** 分别服务于编译过程的不同阶段，体现从高层语义到低层机器的逐步降维。
- 语法制导定义 (**SDD**) 通过属性与语义规则将语义计算与语法分析结合；声明处理需关注类型表达式与存储布局。
- 语法制导翻译 (**SDT**) 将语义动作嵌入产生式，增量翻译避免了冗长的字符串拼接。
- 数组翻译的核心是地址计算，声明从右往左构造类型，寻址从左往右剥离维度。
- 控制流翻译有数值法和短路跳转法；回填技术解决了一遍扫描中跳转目标不确定的问题。
- **switch-case** 根据标签分布选择跳转表、分支链或二分查找策略。

作业

① 数组访问翻译到三地址码

将以下 C 语言代码片段翻译为三地址码 (假设 `int` 宽度为 4, 数组 `a` 为 5×5 的二维数组, 行优先存储):

```
int a[5][5];
int i, j, x;
x = a[i][j] + a[j][i];
```

要求: 写出完整的三地址码序列, 并标注数组元素地址的计算过程。

② 逻辑表达式与控制流回填

给定布尔表达式:

$$(a < b) \parallel (c < d \ \&\& \ e < f)$$

请使用回填技术, 逐步展示其翻译过程, 包括:

- 每条三地址指令的生成顺序。
- 各子表达式的 `truelist` 与 `falselist` 的变化。
- 最终回填完成的完整三地址码。